Six-Family Cricket Shot Recognition with Calibrated Human Review

Hrushikesh Roop Avvari (230249) Voora Rakesh (231174)
Gokavarapu Manikanta Sri Akshith (230418)
Lokesh Kumar Moond (230606) Bollu Sharath chandra (230303)
Indian Institute of Technology Kanpur
EE604, November 2026

Abstract

We study cricket shot recognition with six output families and an explicit human-review decision. The dataset audit covers 10091 CricShot10k clips from fifteen native categories. Cover and straight drives share the Drive family; five non-target categories supply scope-training examples, while three other styles are withheld from development. Frozen video and focused-frame CNN features feed regularised classifiers. Separate source groups provide model tuning, probability calibration and review-threshold selection. A case receives an automatic label only when exactly one calibrated support score exceeds the threshold. On 1070 supported-shot test clips, the selected classifier obtains 75.8% accuracy and 75.2% macro F1. The policy handles 70.0% of supported shots automatically, with 83.8% accuracy among those accepted, and sends 77.8% of 923 non-target clips to review. Deferred cases are not counted as correct. The paired source-group interval supports a positive rejection-rate difference on this benchmark. The experiment reuses earlier benchmark sources, and does not establish universal detection of unusual strokes.

1. Introduction

A useful cricket recogniser should distinguish familiar strokes and admit uncertainty when its supported labels do not explain the input. A short broadcast can contain a conventional shot, an unusual lofted stroke, or a camera following the ball after contact. Assigning the largest of a fixed set of scores always produces a label, even when the movement is poorly represented by the training data.

Our input is one selected batting interval. The six output families are Drive, Defence, Flick, Pull, Square cut and Sweep. Drive combines the source dataset's cover and straight drives. These families reflect common coaching categories [5], rather than a verified worldwide ranking of shot frequency. The alternative outcome is human review, which leaves the model's class unresolved. We do not treat review as a seventh shot category or assume that a subsequent human decision is correct.

The previous eight-class experiment reached approximately 99.0% training accuracy and 73.8% validation accuracy. That gap indicates substantial overfitting. Class count alone does not explain it: the old classifier also had many fitted coefficients relative to its training examples, and videos from new sources differed from its training footage. This rebuild changes the taxonomy, studies regularised heads, and measures automatic coverage together with errors and rejection. Its percentages cannot isolate the effect of reducing the class count.

Added integration. We combine full-video context and ordered focused-frame features with a separately learned scope score. Calibration turns their outputs into support for the six families. A candidate-set rule then decides between an automatic label and human review. Matched confidence-only controls measure the value of the scope component. The underlying CNNs, dimensionality reduction, calibration and reject-option ideas are established methods; the contribution is their specified course-level integration and measured evaluation.

2. Related work and course foundation

Sen et al. [9] use CNN frame features and a GRU for cricket shots. Dihan et al. [1] release CricShot10k and describe batter/bat cropping, segmentation and an EfficientNetV2-S/GRU recogniser. We use their labelled clips, but not their trained cricket classifier or collection detectors. Their reported accuracy uses a different taxonomy and protocol, so it is not a directly comparable baseline.

MC3-18 [10, 11] supplies our full-video representation, while MobileNetV2 [6, 8] reads a larger focused view. Temporal segment representations are established in action

Family	Train	Tune	Cal.	Policy	Test
Drive	711	76	82	78	237
Defence	357	40	42	38	120
Flick	553	67	57	60	184
Pull	444	48	49	52	148
Square cut	608	67	67	68	202
Sweep	537	59	62	58	179

Table 1. Six target-family clip counts. Drive combines cover and straight drives. Review-only clips appear separately in Table 2.

Role	Target	Non-target	Groups
Train	3210	2283	277
Tune	357	258	33
Calibration	359	263	33
Policy	354	265	30
Test	1070	923	109

Table 2. Disjoint source-group roles. Target clips fit the six-family problem; non-target clips fit or evaluate the scope/review problem.

recognition [12]. Camera compensation and optical flow provide a project-level motion measurement [7]; they do not annotate ball contact or batting phases.

Guo et al. [3] motivate calibrating class scores. Selective classification studies errors together with the fraction of inputs a model accepts [2]. Hendrycks et al. [4] motivate learning from auxiliary outlier data. Our small binary scope head uses non-target cricket strokes; it does not reproduce their deep outlier-exposure loss. These sources provide the component ideas, not a guarantee for our data.

The supplied EE604 Lectures 14 part II and 15 cover CNNs and softmax classification. Our feature-to-classifier pipeline follows that foundation. Dimensionality reduction, camera motion, calibration and the review policy are additional project study, with their roles and references stated explicitly.

3. Data and source separation

3.1. Target families and non-target examples

We download all fifteen author archives [1] and audit the actual decoded files. They contain 10091 clips, which differs from the repository summary. We infer no cause for that discrepancy. The audit retains 9965 clips after removing 29 duplicate copies, quarantining 4 clips with conflicting target labels and excluding 93 clips from explicitly described compilation/full-series sources. Another 623 clips belong to the withheld styles and remain unused outside test. Thus 9342 clips enter the five study roles.

The six target families use seven native source categories. Hook, late cut, down the wicket, lofted legside and lofted offside provide non-target examples for the binary scope problem. Reverse sweep, scoop and upper cut never enter training, tuning, calibration or policy selection. Their test cases measure rejection of styles withheld from development. A non-target label means outside the chosen six-family taxonomy, not necessarily an incorrect or non-textbook cricket technique.

Each filename supplies a source-video ID. We keep the source spreadsheet snapshot and inherit the previous source-group links. Exact file/decoded hashes and conservative near-duplicate checks join related sources. The near-duplicate rule uses three pHashes, total Hamming distance at most eight, each distance at most four, duration difference at most 15%, and aligned thumbnail mean absolute error at most five intensity levels. Metadata records differing only by Test day are also joined. The manifest records hashes, group identities and exclusion reasons.

3.2. Five disjoint roles

Training fits all head coefficients, scaling statistics and PCA projections. Tuning chooses the representation, regularisation, neural epoch and scope-head setting. Calibration fits the class temperature and scope sigmoid. Policy groups choose the review threshold. The final test evaluates the fixed system. The inherited validation groups are divided into the three development roles using stratified group folds with seed 604. New source components enter test, and any newly connected component spanning old partitions would be excluded. The completed audit found 0 such bridges.

The test contains 1070 target-family clips and 923 non-target clips from 109 source groups. Existing target-shot test sources were already evaluated in the eight-class study. This is therefore a reused benchmark, not a new independent collection. No current-run test score selects the new heads, calibration or threshold. The added non-target categories expand the rejection study, including styles withheld from development.

Source groups are not independently verified matches. Incomplete descriptions, transformed footage and unidentified replays may remain. Upstream labels have not received a new independent expert annotation study. ImageNet/Kinetics pretraining overlap with every broadcast source has also not been audited.

4. Method

4.1. Context and focused temporal features

We sample 24 positions from the selected interval. The full view is resized to 128×171, centrally cropped to 112×112 and normalised for frozen MC3-18. The resulting spatially pooled map has 24 rows and 512 values per row. Its mean provides global video context.

The focused view crops the original decoded pixels to

$x \in [0.20, 0.80]$, $y \in [0.15, 1.00]$, bounds its longest side to 320 pixels and fits it to 224×224. ImageNet-normalised MobileNetV2 gives 1,280 values per sampled position. This fixed central crop is a broadcast-layout assumption, not a learned batter detector.

With $n(\cdot)$ denoting L2 normalisation, the ordered representation is

$$z = n\big([n(g); n(u); p_1; p_2; p_3]\big), \tag{1}$$

where g and u are complete-interval video/focused means, and p_k is a normalised focused mean from consecutive temporal bin k. It has 5,632 values. Simpler controls use the video mean alone (512 values) or both complete means (1,792 values).

The motion variant pools an 18-position interval from the same feature maps. At 160-pixel analysis width, background Lucas–Kanade tracks and a RANSAC partial-affine fit align adjacent frames before Farneback flow. The 80th percentile of central residual flow measures activity. The highest-activity interval is selected, with full-clip fallback for unreliable fits or weak/flat activity. All parameters and fallback conditions are in the submitted code. Temporal bins and measured activity have no ground-truth contact or phase annotations.

4.2. Regularised heads

Only training data fit standardisation and PCA. Standardised coordinates are clipped to five standard deviations. The initial study retains 128 or 256 whitened PCA components for focused representations. Linear controls use five fixed values of C, from 10^{-5} to 0.1. Small neural heads use 32 or 64 ReLU units, dropout 0.6, AdamW with learning rate 0.001 and weight decay 0.01, class-balanced cross entropy with label smoothing 0.05, and training-feature noise with standard deviation 0.05. Validation never updates these statistics.

Neural training uses batch 64, at most 150 epochs, patience 20 and the earliest epoch with the best tuning macro F1. Two seeds form each recorded neural ensemble. Matched full/motion ensembles are also compared. A development follow-up tests PCA512 with 128 hidden units under the same optimisation settings, adding three ensembles to the original 26 candidates. The initial PCA256 ensemble has 90.8% train and 79.6% tuning accuracy; the larger selected ensemble improves tuning to 82.1% while fitting training almost perfectly. The deployment follows tuning macro F1 across 29 recorded candidates, without a train-plus-tuning refit.

The initial binary logistic scope head uses PCA128 context features and tuning AUROC to choose C. Its tuning AUROC is 0.850, but its strict policy handles only 23.4% of supported development shots. A recorded development-only follow-up compares six small binary neural heads, retaining the linear baseline. The selected full-ordered PCA128/32-unit head reaches 0.886 tuning AUROC at epoch 52. It uses balanced binary cross entropy, smoothing 0.02, and the same dropout/optimiser/noise settings. These follow-ups and the operating preference below were fixed before current-run test outcomes; the reused development sets are not fresh independent evidence.

4.3. Calibrated support and threshold rule

Let p_i be the selected classifier's conditional six-family score and ℓ the binary scope logit. Calibration groups fit temperature T and sigmoid parameters a, b:

$$\widetilde{p}_i = \frac{p_i^{1/T}}{\sum_j p_j^{1/T}}, \qquad q = \sigma(a\ell + b), \qquad s_i = q\widetilde{p}_i. \tag{2}$$

The support scores sum to q, so they need not sum to one. Their interpretation depends on the calibration data mixture; they are not universal probabilities of correct recognition.

For threshold τ, the passing set is $A = \{i : s_i > \tau\}$. Exactly one member gives an automatic label. An empty set or multiple members gives human review. There is no forced top-class fallback. The implementation retains the top candidate separately for inspection while leaving the model's final class null on review cases.

Policy selection sweeps $\tau = 0.10, 0.12, \ldots, 0.90$. The original strict reference seeks maximum known-shot coverage subject to at least 90% accepted-known accuracy and 85% exposure rejection, with at least 50 known automatic cases. It selects 0.66, handling only 38.7% of supported policy cases. The operational requirement prioritises automatic coverage, so the default is fixed at 0.40 after inspecting the development trade-off, before test or the six-class external-clip check. Development goals for this preference are 75% known coverage, 80% accepted-known accuracy and 80% exposure review. At 0.40, policy measurements are 75.7%, 81.7% and 83.0%, respectively. The stricter reference is preserved rather than described as achieved by the default. These are empirical targets, not statistical risk guarantees. The operational development goals are met on policy groups. The default does not meet the original stricter accuracy/rejection goals.

5. Experiments and analysis

5.1. Recognition and overfitting

The selected model is Full+motion N512. Its training accuracy is 99.5%, with 82.1% on tuning and 75.8% on test. A 17.5-point train–tuning gap remains, showing incomplete generalisation despite the smaller regularised head. The taxonomy and role sizes differ from the eight-class study, so this comparison does not prove that fewer classes alone

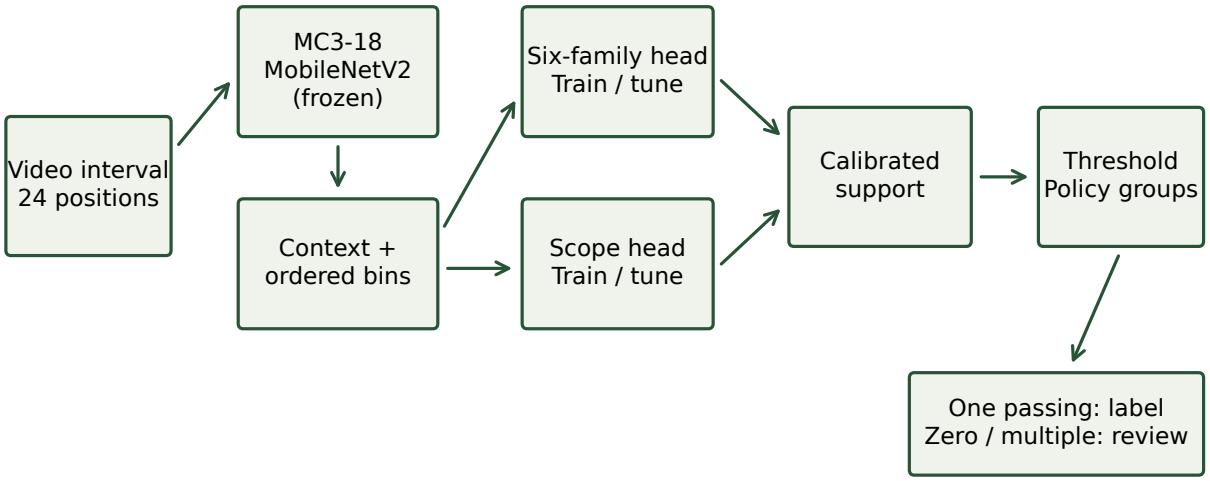


Figure 1. Six-family recognition and explicit review. The classifier and scope heads share frozen context/temporal features, with separately fitted projections. Calibration and threshold selection use separate source groups. Human review leaves the automatic label unresolved; it does not silently retrain the model.

Model	Tune F1	Test acc.	Test F1
Video mean L	64.7	66.4	66.1
Motion bins L	71.7	68.2	67.6
Full bins N128	72.7	69.8	69.1
Motion bins N128	72.8	71.3	70.7
Full+motion N128	74.5	72.2	71.6
Full bins N256	79.0	74.8	74.2
Motion bins N256	77.1	73.8	73.2
Full+motion N256	78.6	76.4	75.7
Full bins N512	78.2	75.2	74.6
Motion bins N512	77.7	73.1	72.5
Full+motion N512	81.4	75.8	75.2

Table 3. Same six-family source split, in percent. L: linear; N128/N256/N512: neural ensembles using that PCA dimension. Complete settings appear in the supplement.

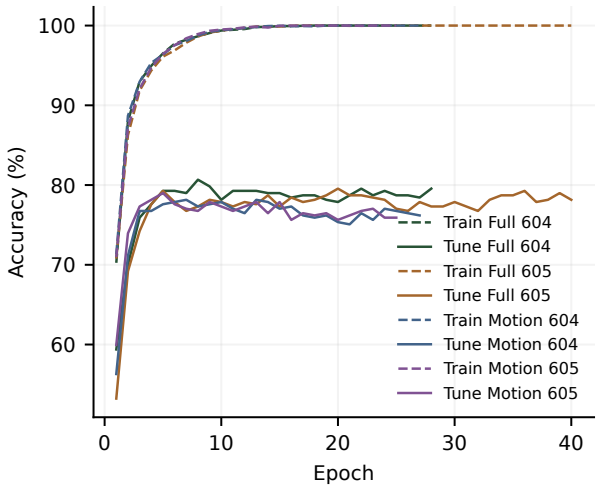


Figure 2. Actual training and tuning traces for the selected neural members, where applicable. Each head's epoch is selected on tuning macro F1. The ensemble's saved predictions determine its reported metrics.

caused an improvement. The same-split raw-video baseline obtains 66.4% test accuracy and 66.1% macro F1. The training-majority baseline obtains 22.1% accuracy.

5.2. Automatic coverage and human review

The fixed threshold is 0.4. Among supported-shot test cases, 749 of 1070 receive automatic labels, with 83.8% accuracy. All 1070 cases still contribute to raw recognition accuracy. The policy defers 77.8% of non-target cases. Automatic accuracy across the complete mixed test set is 65.8%; this counts every non-target automatic label as an error.

The known-shot coverage 95% source-group interval is [64.7, 74.8], and the accepted-known accuracy interval is [80.2, 87.2]. The rejection interval is [74.6, 80.6]. Intervals use 2,000 resamples of whole source groups with seed 604, conditional on the fixed models and grouping assumptions. No rejected case is treated as a correct human answer.

The rejection-rate difference from confidence-only

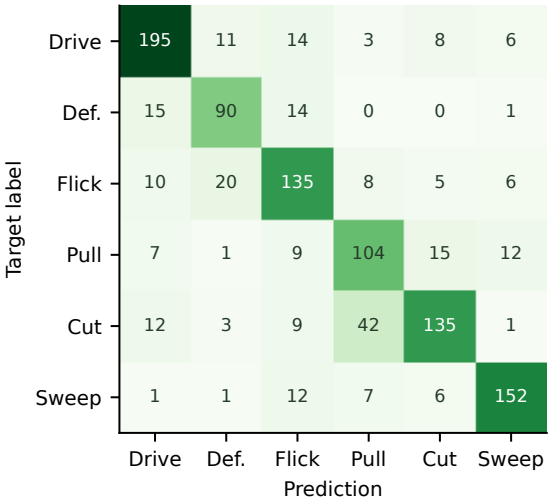


Figure 3. Raw six-family test confusion counts, including cases later deferred by the policy. Rows are target labels; columns are predictions.

Policy	Known C	Known A	Other R	Mixed A
Confidence, same T	91.0	79.7	16.5	44.5
Confidence, matched C	67.3	88.5	55.6	56.4
Scope support	70.0	83.8	77.8	65.8

Table 4. Test percentages. C: automatic coverage; A: accuracy among automatic cases; R: review rate. Matched C refers to coverage matched on policy groups, not on test. Other denotes non-target strokes.

Score	NLL	ECE	Brier
Raw conditional	0.693	0.086	0.343
Temperature scaled	0.655	0.028	0.331

Table 5. Calibration quality on all supported-shot test cases. Lower is better; ECE uses fifteen equal-width bins.

thresholding at policy-matched coverage is +22.21 percentage points, with paired interval [18.2, 26.2]. The paired source-group interval supports a positive rejection-rate difference on this benchmark. The matched-coverage confidence-only threshold is chosen on policy groups, rather than adjusted using test outcomes. Calibration quality is measured using NLL, Brier score and fifteen-bin ECE on held-out data; calibration-set fit quality is not used as an independent result.

5.3. Unknown styles and class-specific workload

Styles withheld from development receive 49.4% review on 158 test clips. This limited result shows that the scope

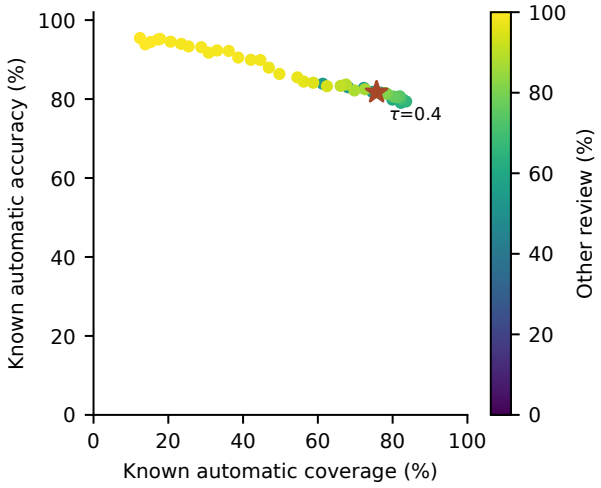


Figure 4. Policy-group coverage and accepted-known accuracy for the fixed threshold grid. Colour indicates non-target review rate. Candidate counting can make coverage non-monotonic in the threshold. The selected point follows development targets.

Non-target type	n	Review
Hook	106	75.5
Late cut	57	64.9
Down the wicket	142	87.3
Lofted legside	246	88.2
Lofted offside	214	85.0
Reverse sweep	45	24.4
Scoop	67	58.2
Upper cut	46	60.9

Table 6. Non-target test review rates in percent. Reverse sweep, scoop and upper cut were withheld from all development roles.

model does not reliably reject arbitrary new styles, despite stronger aggregate rejection of exposure categories. There are 205 non-target automatic labels, all counted as errors in mixed automatic accuracy. The supplement gives each target family's automatic coverage and accepted accuracy, so a favourable overall figure cannot hide a class that is mostly deferred.

5.4. Implementation and demo

The local application uses ONNX encoders and numeric NumPy heads. It loads no pickled model objects. Training extraction and runtime preprocessing are checked for numerical agreement. The single-pass interval decoder reproduces the training full/focused pixels on lossless synthetic checks, and fresh real-clip replays verify deployed scores and decisions.

Uploads may be up to 100 MB and two minutes, with one selected interval of at most five seconds, 600 frames

and roughly 4K resolution. Training clips are much tighter, usually around one second. The interface shows six support scores, the fixed threshold, sampled inputs and measured activity. Review cases have no automatic class. A reviewer can record a label or "outside/unclear" locally; that record does not update the model. Warm-encoder replay of 10 real examples has median 0.70 seconds and 95th percentile 0.80 seconds on the recorded test machine. This includes core interval decoding and inference, but excludes model initialisation, JPEG preview encoding and browser rendering.

6. Limitations and conclusion

This system covers six broad families from a short-broadcast dataset. Drive no longer distinguishes cover from straight drives. A hook can be conventional cricket but still lie outside the supported taxonomy. Scope membership is therefore different from textbook technique, and a confident unfamiliar stroke can still be accepted incorrectly.

Source descriptions and upstream labels remain imperfect. Fixed central crops can miss an off-centre batter. Frozen encoders limit adaptation to new cameras and batting variations. The reused benchmark and repeated development-set selection constrain the strength of generalisation claims. A new independently annotated match collection is needed for a stronger external evaluation.

The course extension combines focused/global temporal context, calibrated scope support and an explicit candidate-set review rule. The paired source-group interval supports a positive rejection-rate difference on this benchmark. The measured coverage, automatic errors and non-target rejection describe the trade-off; a threshold cannot guarantee both near-complete automation and correct review of every unusual shot.

7. Individual contributions

Hrushikesh Roop Avvari (230249). Worked on feature extraction, classifier implementation and model training. Compared the model configurations and analysed training behaviour and overfitting.

Voora Rakesh (231174). Contributed to evaluating the model by checking the confusion matrix and per-class precision, recall and F1 scores. Cross-checked the result tables against saved experiment outputs and prepared the literature review and results slides.

Gokavarapu Manikanta Sri Akshith (230418). Worked on dataset preparation, class mapping and experimental evaluation. Checked the reported results and contributed to writing and reviewing the report.

Lokesh Kumar Moond (230606). Contributed to application testing by checking video uploads, interval selection and threshold-based human-review decisions. Verified exported prediction and review records using successful and failed examples, and prepared the live demonstration.

Bollu Sharath chandra (230303). Worked on the threshold-based human-review mechanism and the application interface. Tested video uploads, prediction outputs and the review workflow.

References

[1] Mubtasim Kamal Dihan, Abdullah, Amina, and Sabbir Ahmed. CricShot10k: A large-scale video dataset for cricket shot classification. *IEEE Access*, 14:23428–23447, 2026.

[2] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *NeurIPS*, 2017.

[3] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, pages 1321–1330, 2017.

[4] Dan Hendrycks, Mantas Mazeika, and Thomas Dietterich. Deep anomaly detection with outlier exposure. arXiv:1812.04606, 2018.

[5] New Zealand Cricket. Sample coaching plan: Skill sets. https://www.nzc.nz/media/15726/sample-coaching-plan-skill-sets.pdf. Accessed November 2026.

[6] ONNX Model Zoo contributors. MobileNetV2 pretrained onnx models. Accessed 17 October 2026.

[7] OpenCV contributors. Optical flow: Lucas–kanade and farneback methods. Accessed November 2026.

[8] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *CVPR*, pages 4510–4520, 2018.

[9] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba. CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit. *Sensors*, 21(8):2846, 2021.

[10] TorchVision contributors. MC3-18: Official model and kinetics-400 weights. Accessed November 2026.

[11] Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. A closer look at spatiotemporal convolutions for action recognition. In *CVPR*, 2018.

[12] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. Temporal segment networks: Towards good practices for deep action recognition. In *ECCV*, 2016.

A. Complete candidates and workload

Candidate	C / epochs	Train acc.	Tune F1	Test acc.	Test F1
Video mean L	C=1e-05	41.5	38.1	41.3	39.5
Video mean L	C=0.0001	51.2	48.9	48.4	47.1
Video mean L	C=0.001	66.0	61.3	61.0	60.2
Video mean L	C=0.01	77.5	64.7	66.4	66.1
Video mean L	C=0.1	87.1	63.4	65.0	64.8
Context means L	C=1e-05	69.1	66.3	62.0	61.3
Context means L	C=0.0001	70.0	67.9	63.9	63.2
Context means L	C=0.001	71.0	68.0	64.4	63.7
Context means L	C=0.01	73.8	67.7	64.5	63.7
Context means L	C=0.1	75.5	66.4	65.2	64.5
Full bins L	C=1e-05	71.3	69.1	63.9	63.2
Full bins L	C=0.0001	72.9	70.4	65.2	64.4
Full bins L	C=0.001	74.5	71.0	65.8	65.2
Full bins L	C=0.01	76.3	68.9	68.1	67.6
Full bins L	C=0.1	77.9	68.0	69.0	68.5
Motion bins L	C=1e-05	73.2	70.4	67.5	66.9
Motion bins L	C=0.0001	74.1	70.8	68.7	68.0
Motion bins L	C=0.001	75.3	71.7	68.2	67.6
Motion bins L	C=0.01	78.1	68.8	68.9	68.3
Motion bins L	C=0.1	79.0	69.9	69.3	68.6
Full bins N128	17/15	80.4	72.7	69.8	69.1
Motion bins N128	38/20	83.6	72.8	71.3	70.7
Full+motion N128	17/15/38/20	83.1	74.5	72.2	71.6
Full bins N256	9/11	90.8	79.0	74.8	74.2
Motion bins N256	33/8	95.6	77.1	73.8	73.2
Full+motion N256	9/11/33/8	94.2	78.6	76.4	75.7
Full bins N512	8/20	99.8	78.2	75.2	74.6
Motion bins N512	7/5	98.0	77.7	73.1	72.5
Full+motion N512	8/20/7/5	99.5	81.4	75.8	75.2

Table 7. All fixed classifier candidates. Percentages include every known-shot case. The deployment follows tuning macro F1, not test ranking.

Family	n	Raw precision	Raw recall	Raw F1	Auto coverage	Auto accuracy
Drive	237	81.2	82.3	81.8	72.6	89.0
Defence	120	71.4	75.0	73.2	81.7	80.6
Flick	184	69.9	73.4	71.6	80.4	81.8
Pull	148	63.4	70.3	66.7	56.1	83.1
Square cut	202	79.9	66.8	72.8	56.9	73.9
Sweep	179	85.4	84.9	85.2	74.3	91.0

Table 8. Per-family test performance and review workload, in percent. A dash means that no automatic cases exist for that denominator.

Role	Scope AUROC
Train	0.950
Tune	0.886
Calibration	0.869
Policy	0.859
Test	0.807

Table 9. Scope discrimination across roles. These values concern the chosen taxonomy and dataset mixture.

Quantity	Lower / upper
Raw known accuracy	[71.8, 79.6]
Raw known macro f1	[70.6, 79.0]
Known coverage	[64.7, 74.8]
Known automatic accuracy	[80.2, 87.2]
Automatic accuracy all	[60.5, 71.2]
Out of scope review rate	[74.6, 80.6]
Review gain over confidence matched coverage	[18.2, 26.2]

Table 10. Source-group bootstrap 95% intervals, in percent or percentage points for the paired difference. All are conditional on the fixed models and grouping assumptions.

B. Code, data and source links

Project repository: https://github.com/hrushikeshroop/cricket-shot-recognition-ee604. The following source appendix contains every submitted implementation, configuration, dependency and test source file. Numeric model weights and saved individual probabilities accompany the repository.

Dataset: https://github.com/Dihan69/CricShot10k. The authors specify non-commercial research use with citation; original footage rights remain with the respective owners. The repository provides the public archive URLs, checksums and source-video spreadsheet snapshot. The demo uses real decoded dataset frames.

Pretrained sources: TorchVision MC3-18 and ONNX Model Zoo MobileNetV2. Component source links and licences appear in `docs/sources.md` and `THIRD_PARTY_NOTICES.md`. No pretrained cricket classifier is imported.

Software: NumPy, OpenCV, ONNX Runtime, Pillow, Flask, PyTorch, scikit-learn, SciPy, pytest and gdown. The source appendix includes the tested dependency versions.

Report template: CVPR 2022 author kit supplied in the course email. The report uses its unmodified archived style. Component source links and licences also accompany the project repository.

Original code and configuration. Line numbers identify source lines. Wrapped lines continue without a number.

app.py

```
"""Local six-family demo: one supported prediction or an explicit review request."""
from pathlib import Path
import sys,base64,io,json,tempfile,threading
import numpy as np
from PIL import Image
from flask import Flask,jsonify,render_template,request,send_file
from werkzeug.exceptions import RequestEntityTooLarge
ROOT=Path(__file__).resolve().parent;sys.path.insert(0,str(ROOT/'src'))
from cricketshot.recognizer import Recognizer


def create_app(recognizer=None):
    app=Flask(__name__);app.config['MAX_CONTENT_LENGTH']=100*1024*1024;holder=[recognizer];lock=threading.Lock()
    def examples():return json.loads((ROOT/'demo/examples.json').read_text())
    def find(identifier):return next((e for e in examples() if e['id']==identifier),None)
    def infer(path,start=0.,end=None):
        with lock:
            if holder[0] is None:holder[0]=Recognizer()
            result,frames=holder[0].predict(path,start,end)
        result['previews']={}
        for name,sequence in frames.items():
            previews=[]
            for i in np.rint(np.linspace(0,len(sequence)-1,8)).astype(int):
                im=Image.fromarray(sequence[i]);im.thumbnail((240,160));buf=io.BytesIO();im.save(buf,format='JPEG',quality=85);previews.append(''+base64.b64encode(buf.getvalue()).decode('ascii'))
            result['previews'][name]=previews
        return result
    @app.context_processor
    def identity():return {'submission':json.loads((ROOT/'configs/submission.json').read_text())}
    @app.get('/')
    @app.get('/video')
    def index():return render_template('index.html')
    @app.get('/api/status')
    def status():return jsonify(ready=all((ROOT/'models'/f).is_file() for f in ['classifier.json','heads.npz','scope_head.npz','mc3_features.onnx','mobilenet_features.onnx']),version='4.0.0',classes=6)
    @app.get('/api/evidence')
    def evidence():return jsonify(json.loads((ROOT/'results/metrics.json').read_text()))
    @app.get('/api/examples')
    def example_list():return jsonify([{k:v for k,v in e.items() if k not in {'path','thumbnail'}} for e in examples()])
    @app.get('/api/example/<identifier>/thumbnail')
    def thumbnail(identifier):
        e=find(identifier);return send_file(ROOT/e['thumbnail'],mimetype='image/jpeg') if e else (jsonify(error='Example not found'),404)
    @app.post('/api/example/<identifier>')
    def example(identifier):
        e=find(identifier)
        if not e:return jsonify(error='Example not found'),404
        result=infer(ROOT/e['path']);result['example']={k:v for k,v in e.items() if k not in {'path','thumbnail'}};return jsonify(result)
    @app.post('/api/predict')
    def predict():
        upload=request.files.get('video')
        if not upload or not upload.filename:return jsonify(error='Choose a video first.'),400
        suffix=Path(upload.filename).suffix.lower()
        if suffix not in {'.avi','.mp4','.mov','.mkv','.webm'}:return jsonify(error='Use AVI, MP4, MOV, MKV or WebM.'),400
        start=float(request.form.get('start') or 0);end=float(request.form['end']) if request.form.get('end') else None
        with tempfile.TemporaryDirectory(prefix='cricketshot6_') as folder:
            path=Path(folder)/('clip'+suffix);upload.save(path);return jsonify(infer(path,start,end))
```

app.py

```
    @app.errorhandler(ValueError)
    def invalid(exc):return jsonify(error=str(exc)),400
    @app.errorhandler(FileNotFoundError)
    def missing(exc):return jsonify(error='Project files are missing. Follow README setup.'),503
    @app.errorhandler(RequestEntityTooLarge)
    def too_large(exc):return jsonify(error='Upload below 100 MB, then select a batting interval up to five seconds.'),413
    return app


if __name__=='__main__':
    import argparse

parser=argparse.ArgumentParser();parser.add_argument('--port',type=int,default=6040);args=parser.parse_args()
    create_app().run(host='127.0.0.1',port=args.port,debug=False,threaded=True)
```

configs/class_followup.json

```
{
  "reason": "The initial regularised six-family classifier reaches 79.6% tuning accuracy. Before opening current-run test results, test whether retaining more frozen-feature dimensions improves generalisation and the automatic-review trade-off.",
  "pca": 512,
  "hidden": 128,
  "seeds": [604,605],
  "representations": ["full_ordered","motion_ordered"],
  "unchanged": "Train-only fitting, neural optimisation settings, earliest best tune macro-F1 epoch, original 26 candidates, source roles and test withholding. No user clip selection.",
  "selection": "Highest tuning macro F1 among the original and three additional two-/four-head ensembles. No refit on tuning groups."
}
```

configs/experiment.json

```
{
  "version": 1,
  "seed": 604,
  "created_utc": "2026-11-05T21:42:39.193443+00:00",
  "task": "Six common shot families with automatic classification or human review",
  "classes": [
    "drive",
    "defence",
    "flick",
    "pull",
    "square_cut",
    "sweep"
  ],
  "source_mapping": {
    "cover_drive": "drive",
    "straight_drive": "drive",
    "defence": "defence",
    "flick": "flick",
    "pull": "pull",
    "square_cut": "square_cut",
    "sweep": "sweep"
  },
  "exposure_classes": [
    "hook",
    "late_cut",
    "down_the_wicket",
    "lofted_legside",
    "lofted_offside"
```

configs/experiment.json

```
  ],
  "unseen_style_classes": [
    "reverse_sweep",
    "scoop",
    "upper_cut"
  ],
  "taxonomy_note": "Common coaching families, not an empirically verified worldwide frequency ranking. Drive combines cover and straight drives; six output families are not six native archive labels.",
  "data": "CricShot10k author archives, all 15 native categories; audit actual files and cross-category duplicates.",
  "excluded_source_ids": [
    20,
    29,
    40,
    56
  ],
  "split_rule": "Preserve previous train/test source separation. Split validation components deterministically into disjoint tune, calibration and policy roles. Exclude newly connected components that bridge old train/validation/test partitions. New source components enter test. Unseen-style classes never enter training, tuning, calibration or policy selection.",
  "test_history": "Existing test source groups were evaluated in the eight-class study. The new six-class test is a reused benchmark, not a wholly new independent benchmark. No current-run test scores will select models/calibration/policy.",
  "representations": [
    "video_mean",
    "context_mean",
    "full_ordered",
    "motion_ordered"
  ],
  "linear_C": [
    1e-05,
    0.0001,
    0.001,
    0.01,
    0.1
  ],
  "linear": "Train-only standardisation; raw video-mean baseline, other descriptors use train-only whitened PCA128; balanced multinomial logistic regression.",
  "neural_configs": [
    {
      "pca": 128,
      "hidden": 32
    },
    {
      "pca": 256,
      "hidden": 64
    }
  ],
  "neural_representations": [
    "full_ordered",
    "motion_ordered"
  ],
  "neural_seeds": [
    604,
    605
  ],
  "neural": {
    "dropout": 0.6,
    "weight_decay": 0.01,
    "learning_rate": 0.001,
    "label_smoothing": 0.05,
    "feature_noise_std": 0.05,
    "max_epochs": 150,
    "patience": 20,
```

configs/experiment.json

```
    "batch_size": 64,
    "input_clip": 5.0
  },
  "model_selection": "Tune macro F1; evaluate recorded linear candidates, two-seed neural ensembles and matched full/motion ensembles. No tune-plus-train refit.",
  "scope_head": "Binary balanced logistic regression on train-only PCA128 context features; known six families versus the five exposure categories. Select C using tune AUROC.",
  "calibration": "Fit one temperature for the six-class conditional distribution and a Platt sigmoid for scope logits using calibration groups only.",
  "support": "Joint support_i = calibrated in-scope score times calibrated conditional class score_i. Scores sum to the in-scope score, not necessarily one.",
  "review_rule": "Exactly one support score strictly above the selected common threshold -> automatic label; zero or multiple -> human review. No forced top-1 fallback.",
  "policy_grid": [
    0.1,
    0.12,
    0.14,
    0.16,
    0.18,
    0.2,
    0.22,
    0.24,
    0.26,
    0.28,
    0.3,
    0.32,
    0.34,
    0.36,
    0.38,
    0.4,
    0.42,
    0.44,
    0.46,
    0.48,
    0.5,
    0.52,
    0.54,
    0.56,
    0.58,
    0.6,
    0.62,
    0.64,
    0.66,
    0.68,
    0.7,
    0.72,
    0.74,
    0.76,
    0.78,
    0.8,
    0.82,
    0.84,
    0.86,
    0.88,
    0.9
  ],
  "policy_targets": {
    "known_automatic_accuracy": 0.9,
    "known_coverage_goal": 0.75,
    "exposure_review_rate": 0.85
  },
  "policy_selection": "On policy groups only: maximise known coverage among thresholds meeting 90% known automatic accuracy and 85% exposure review. Report whether the 75% known-coverage goal is met. If no threshold meets the accuracy/rejection targets, choose the greatest balanced objective (known coverage * known automatic
```

configs/experiment.json

```
    accuracy - unknown false-automatic rate); explicitly report the target failure.",
142   "uncertainty": "2,000 source-component bootstrap draws, seed604, fixed models and policy; no independent-match
     or universal unknown-detection guarantee.",
143   "external_clip": "User Kohli clip is a previously seen qualitative diagnostic only. No threshold, training
    label or model choice is fit to that clip.",
144   "novelty": "Course-level integration of focused/global temporal context, calibrated scope support, explicit
    candidate-set review and measured risk/coverage; established component methods attributed."
145 }
```

configs/operating_point.json

```
1  {
2    "threshold": 0.4,
3    "selection_role": "policy",
4    "selection_basis": "The coverage-oriented operating point was selected from two development-only trade-offs
    before current-run test evaluation and the external-clip diagnostic.",
5    "development_goals": {"known_coverage":0.75,"known_automatic_accuracy":0.8,"exposure_review_rate":0.8},
6    "cautious_reference_threshold": 0.66,
7    "note": "This changes the operating preference, not the fitted classifier or calibration. Original 90%
    accuracy / 85% rejection goals remain recorded as a stricter reference. No statistical error guarantee."
8  }
```

configs/scope_experiment.json

```
1  {
2    "version": 2,
3    "reason": "The initial linear scope model reaches 90% known automatic accuracy but only 23.4% known coverage
    on policy groups. This development-only follow-up tests nonlinear taxonomy membership before any current-run
    test scores are opened.",
4    "fit_role": "train",
5    "selection_role": "tune",
6    "selection_metric": "binary AUROC; initial linear model remains a candidate",
7    "representations": [{"name":"context_mean","pca":128,"hidden":32},{"name":"full_ordered","pca":128,"hidden":32
    },{"name":"full_ordered","pca":256,"hidden":64}],
8    "seeds": [604,605],
9    "dropout": 0.6,
10   "weight_decay": 0.01,
11   "learning_rate": 0.001,
12   "feature_noise_std": 0.05,
13   "label_smoothing": 0.02,
14   "batch_size": 64,
15   "max_epochs": 150,
16   "patience": 20,
17   "unchanged": "Class heads, data roles, unseen-style withholding, calibration roles, common threshold grid and
    review rule. The user clip is not used for selection."
18 }
```

configs/submission.json

```
1  {
2    "group_name": "EE604 Project Group",
3    "github_url": "https://github.com/hrushikeshroop/cricket-shot-recognition-ee604",
4    "member_names": [
5      "Hrushikesh Roop Avvari",
6      "Voora Rakesh",
7      "Gokavarapu Manikanta Sri Akshith",
8      "Lokesh Kumar Moond",
9      "Bollu Sharath chandra"
10   ],
11   "members": [
12     {
```

configs/submission.json

```
      "name": "Hrushikesh Roop Avvari",
      "roll_number": "230249",
      "contribution": "Worked on feature extraction, classifier implementation and model training. Compared the model configurations and analysed training behaviour and overfitting."
    },
    {
      "name": "Voora Rakesh",
      "roll_number": "231174",
      "contribution": "Contributed to evaluating the model by checking the confusion matrix and per-class precision, recall and F1 scores. Cross-checked the result tables against saved experiment outputs and prepared the literature review and results slides."
    },
    {
      "name": "Gokavarapu Manikanta Sri Akshith",
      "roll_number": "230418",
      "contribution": "Worked on dataset preparation, class mapping and experimental evaluation. Checked the reported results and contributed to writing and reviewing the report."
    },
    {
      "name": "Lokesh Kumar Moond",
      "roll_number": "230606",
      "contribution": "Contributed to application testing by checking video uploads, interval selection and threshold-based human-review decisions. Verified exported prediction and review records using successful and failed examples, and prepared the live demonstration."
    },
    {
      "name": "Bollu Sharath chandra",
      "roll_number": "230303",
      "contribution": "Worked on the threshold-based human-review mechanism and the application interface. Tested video uploads, prediction outputs and the review workflow."
    }
  ]
}
```

pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "cricketshot-ee604"
version = "4.0.0"
description = "Six cricket shot families with calibrated support and human review"
requires-python = ">=3.11"
dependencies = ["numpy>=1.26", "Pillow>=10", "onnxruntime>=1.17", "opencv-python-headless>=4.9", "Flask>=3.0"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

requirements-dev.txt

```
# Install the official CPU torch/torchvision wheels first, as shown in README.
-r requirements-lock.txt
torch==2.15.0
torchvision==0.29.2
scikit-learn==1.9.1
scipy==1.18.1
```

requirements-dev.txt

```
ImageHash==4.3.2
onnx==1.23.2
pytest==9.2.0
requests==2.34.2
threadpoolctl==3.8.0
tqdm==4.70.1
gdown==6.5.1
```

requirements-lock.txt

```
# Tested runtime: Windows x64, Python 3.14.5. Training dependencies are separate.
blinker==1.9.0
click==8.5.0
flask==3.1.3
flatbuffers==25.12.19
itsdangerous==2.2.0
jinja2==3.1.6
markupsafe==3.0.3
numpy==2.5.4
onnxruntime==1.31.1
opencv-python-headless==5.0.0.93
packaging==26.3
pillow==13.0.0
protobuf==7.36.2
werkzeug==3.1.9
```

requirements.txt

```
# Demo and inference, Python 3.11 or newer. See requirements-lock.txt for tested versions.
numpy>=1.26
Pillow>=10
onnxruntime>=1.17
opencv-python-headless>=4.9
Flask>=3.0
```

run_demo.bat

```
@echo off
cd /d "%~dp0"
if not exist ".venv\Scripts\python.exe" (
  echo Run setup.bat first.
  pause
  exit /b 1
)
echo CricketShot will be available at http://127.0.0.1:6040
".venv\Scripts\python.exe" app.py
pause
```

scripts/calibrate.py

```
"""Calibrate on dedicated groups, then select review threshold on policy groups."""
from pathlib import Path
import argparse,json,time
import numpy as np
from scipy.optimize import minimize_scalar
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from threadpoolctl import threadpool_limits
from study import ROOT,read,sha,data,candidate_probabilities,selective,nll_ece
from cricketshot.policy import softmax,head_logits,calibrated_support,sigmoid
```

scripts/calibrate.py

```
 11
 12
 13  def main():
 14
    parser=argparse.ArgumentParser();parser.add_argument('--output-dir',default='results');args=parser.parse_args()
 15      out=ROOT/args.output_dir;selection=json.loads((out/'selection.json').read_text());cfg=read('configs/experime
    nt.json')
 16      assert selection['scope_fitted'],'Fit the scope head before calibration.'
 17      if selection['test_accessed_during_selection'] or (out/'metrics.json').exists() or
    (out/'policy.json').exists():raise RuntimeError('Do not overwrite a frozen/evaluated review policy.')
 18      rows,x,y,splits,groups=data();probabilities=candidate_probabilities(selection,x,out);p=probabilities[selecti
    on['deployment']]
 19      calibration=(splits=='calibration');known_calibration=calibration&(y>=0);policy=splits=='policy'
 20      assert not (set(groups[calibration])&set(groups[policy]))
 21      logp=np.log(np.maximum(p,1e-12))
 22      def objective(log_temperature):
 23          q=softmax(logp[known_calibration]/np.exp(log_temperature))
 24          return float(-np.log(np.maximum(q[np.arange(len(q)),y[known_calibration]],1e-12)).mean())
 25      fitted=minimize_scalar(objective,bounds=(np.log(.05),np.log(20)),method='bounded')
 26      if not fitted.success:raise RuntimeError('Temperature optimisation failed.')
 27      with np.load(out/'scope_head.npz',allow_pickle=False) as z:h={k:z[k].copy() for k in z.files}
 28      with threadpool_limits(limits=4):scope_logits=head_logits(x[selection.get('scope_representation','context_me
    an')],h).ravel()
 29      platt=LogisticRegression(C=100,max_iter=3000,tol=1e-8)
 30      with threadpool_limits(limits=4):platt.fit(scope_logits[calibration,None],(y[calibration]>=0).astype(int))
 31      record={'temperature':float(np.exp(fitted.x)),'scope_slope':float(platt.coef_[0,0]),'scope_intercept':float(
    platt.intercept_[0]),'role':'calibration','n':int(calibration.sum()),'known':int(known_calibration.sum()),'selec
    tion_sha256':sha(out/'selection.json'),'calibration_code_sha256':sha(Path(__file__))}
 32      support,conditional,scope=calibrated_support(p,scope_logits,record)
 33      grid=[];targets=cfg['policy_targets']
 34      for threshold in cfg['policy_grid']:
 35          measured=selective(y[policy],support[policy],threshold)
 36          measured['threshold']=threshold
 37          measured['eligible']=bool(measured['automatic_known']>=50 and
    measured['known_automatic_accuracy']>=targets['known_automatic_accuracy'] and
    measured['out_of_scope_review_rate']>=targets['exposure_review_rate'])
 38          measured['fallback_objective']=float(measured['known_coverage']*(measured['known_automatic_accuracy'] or
     0)-(1-measured['out_of_scope_review_rate']))
 39          grid.append(measured)
 40      eligible=[r for r in grid if r['eligible']]
 41      chosen=max(eligible,key=lambda r:(r['known_coverage'],r['known_automatic_accuracy'],-r['threshold'])) if
    eligible else max(grid,key=lambda r:(r['fallback_objective'],r['known_automatic_accuracy'] or
    0,-r['threshold']))
 42      fixed={'threshold':chosen['threshold'],'role':'policy','policy_metrics':chosen,'accuracy_and_rejection_targe
    ts_met':bool(eligible),'known_coverage_goal_met':chosen['known_coverage']>=targets['known_coverage_goal'],'targe
    ts':targets,'deployment':selection['deployment'],'grid':grid,'support_definition':cfg['support'],'decision_rule'
    :cfg['review_rule'],'policy_code_sha256':sha(ROOT/'src/cricketshot/policy.py'),'calibration':record,'finalized_u
    tc':time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime())}
 43      fixed['policy_calibration_quality']={'raw':nll_ece(y[policy&(y>=0)],p[policy&(y>=0)]),'temperature_scaled':n
    ll_ece(y[policy&(y>=0)],conditional[policy&(y>=0)]),'scope_auroc':float(roc_auc_score((y[policy]>=0).astype(int)
    ,scope[policy]))}
 44      fixed['scope_auroc_by_role']={s:float(roc_auc_score((y[splits==s]>=0).astype(int),scope[splits==s])) for s
    in ['train','tune','calibration','policy']}
 45      # Hold the chosen threshold fixed while measuring the confidence-only ablation.
 46      fixed['confidence_only_same_threshold']=selective(y[policy],conditional[policy],chosen['threshold'])
 47      baseline_grid=[dict(selective(y[policy],conditional[policy],t),threshold=t) for t in cfg['policy_grid']]
 48      matched=min(baseline_grid,key=lambda
    r:(abs(r['known_coverage']-chosen['known_coverage']),-(r['known_automatic_accuracy'] or 0),r['threshold']))
 49      fixed['confidence_only_matched_coverage']={'threshold':matched['threshold'],'policy_metrics':matched,'select
    ion':'Closest known-shot coverage to the main policy on policy groups only; ties prefer higher automatic known
    accuracy, then lower threshold.'}
 50      (out/'calibration.json').write_text(json.dumps(record,indent=2));(out/'policy.json').write_text(json.dumps(f
    ixed,indent=2))
 51      print(json.dumps({k:fixed[k] for k in ['threshold','policy_metrics','accuracy_and_rejection_targets_met','kn
```

scripts/calibrate.py

```
    own_coverage_goal_met','policy_calibration_quality']},indent=2),flush=True)
52
53
54  if __name__=='__main__':main()
```

scripts/evaluate.py

```
 1  """Evaluate frozen models and review policies once, then export numeric deployment files."""
 2  from pathlib import Path
 3  import argparse,json,time
 4  import numpy as np
 5  from sklearn.metrics import roc_auc_score
 6  from threadpoolctl import threadpool_limits
 7  from study import ROOT,read,sha,data,metrics,candidate_probabilities,selective,nll_ece
 8  from cricketshot.policy import CLASSES,head_logits,calibrated_support
 9  from export_model import export
10
11
12  def bootstrap(y,groups,conditional,support,threshold,baseline_threshold,seed=604,draws=2000):
13      rng=np.random.default_rng(seed);ids=np.unique(groups);members={g:np.flatnonzero(groups==g) for g in
    ids};values=[]
14      for _ in range(draws):
15          idx=np.concatenate([members[g] for g in rng.choice(ids,len(ids),replace=True)]);known=y[idx]>=0
16          main=selective(y[idx],support[idx],threshold);base=selective(y[idx],conditional[idx],baseline_threshold)
17          raw=metrics(y[idx][known],conditional[idx][known].argmax(axis=1))
18          values.append([raw['accuracy'],raw['macro_f1'],main['known_coverage'],main['known_automatic_accuracy']
    if main['known_automatic_accuracy'] is not None else np.nan,main['automatic_accuracy_all'] if
    main['automatic_accuracy_all'] is not None else
    np.nan,main['out_of_scope_review_rate'],main['out_of_scope_review_rate']-base['out_of_scope_review_rate']])
19      keys=['raw_known_accuracy','raw_known_macro_f1','known_coverage','known_automatic_accuracy','automatic_accur
    acy_all','out_of_scope_review_rate','review_gain_over_confidence_matched_coverage']
20      a=np.asarray(values);return {k:[float(v) for v in np.nanquantile(a[:,i],[.025,.975])] for i,k in
    enumerate(keys)}
21
22
23  def main():
24      parser=argparse.ArgumentParser();parser.add_argument('--output-dir',default='results');parser.add_argument('
    --deploy',action='store_true');args=parser.parse_args()
25      out=ROOT/args.output_dir;selection=json.loads((out/'selection.json').read_text());fixed=json.loads((out/'pol
    icy.json').read_text());cfg=read('configs/experiment.json')
26      if selection['test_accessed_during_selection'] or (out/'metrics.json').exists():raise RuntimeError('Frozen
    test evidence already exists; do not overwrite it.')
27      assert selection['protocol_sha256']==sha(ROOT/'configs/experiment.json') and
    selection['manifest_sha256']==sha(ROOT/'data/manifest.csv')
28      assert fixed['policy_code_sha256']==sha(ROOT/'src/cricketshot/policy.py')
29      rows,x,y,splits,groups=data(include_test=True);values=candidate_probabilities(selection,x,out);p=values[sele
    ction['deployment']]
30      with np.load(out/'scope_head.npz',allow_pickle=False) as z:scope_head={k:z[k].copy() for k in z.files}
31      with threadpool_limits(limits=4):scope_logits=head_logits(x[selection.get('scope_representation','context_me
    an')],scope_head).ravel()
32      support,conditional,scope=calibrated_support(p,scope_logits,fixed['calibration']);threshold=fixed['threshold
    '];test=splits=='test';known=y>=0
33      models={}
34      for name in selection['candidates']:
35          models[name]={s:metrics(y[(splits==s)&known],values[name][(splits==s)&known].argmax(axis=1)) for s in
    ['train','tune','calibration','policy','test']}
36      triage={s:selective(y[splits==s],support[splits==s],threshold) for s in
    ['train','tune','calibration','policy','test']}
37      native=np.array([r['label'] for r in rows]);unseen=np.isin(native,cfg['unseen_style_classes'])
38      unknown_details={label:selective(y[test&(native==label)],support[test&(native==label)],threshold) for label
    in cfg['exposure_classes']+cfg['unseen_style_classes']}
39      baseline_t=fixed['confidence_only_matched_coverage']['threshold']
40      intervals=bootstrap(y[test],groups[test],conditional[test],support[test],threshold,baseline_t,cfg['seed'])
```

scripts/evaluate.py

```
    result={'classes':CLASSES,'deployment':selection['deployment'],'threshold':threshold,'models':models,'triage':triage,'unknown_by_native_label':unknown_details,'unseen_style_triage':selective(y[test&unseen],support[test&unseen],threshold),'exposure_style_triage':selective(y[test&~unseen&~known],support[test&~unseen&~known],threshold),
            'confidence_only_same_threshold':selective(y[test],conditional[test],threshold),'confidence_only_matched_coverage':{'threshold':baseline_t,'test':selective(y[test],conditional[test],baseline_t)},
            'calibration_quality':{'raw_test':nll_ece(y[test&known],p[test&known]),'scaled_test':nll_ece(y[test&known],conditional[test&known]),'scope_test_auroc':float(roc_auc_score(known[test].astype(int),scope[test]))},
            'source_group_95ci':intervals,'bootstrap_draws':2000,'bootstrap_seed':cfg['seed'],'test_group_count':int(len(np.unique(groups[test]))),'test_history':cfg['test_history'],'policy_targets':cfg['policy_targets'],'policy_target_status':{k:fixed[k] for k in ['accuracy_and_rejection_targets_met','known_coverage_goal_met']},'evaluation_code_sha256':sha(Path(__file__)),'evaluated_utc':time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime())}
    counts=np.bincount(y[(splits=='train')&known],minlength=6);majority=int(counts.argmax());result['training_majority_baseline']=metrics(y[test&known],np.full(int((test&known).sum()),majority))
    evidence={'y_true':y,'split':splits,'group':groups,'native_label':native,'path':np.array([r['path'] for r in rows]),'scope_logits':scope_logits,'scope_score':scope,'conditional_raw':p,'conditional_calibrated':conditional,'support':support}
    for name in selection['candidates']:evidence[name+'__probabilities']=values[name]
    np.savez_compressed(out/'predictions.npz',**evidence)
    export(out,args.deploy);(out/'metrics.json').write_text(json.dumps(result,indent=2))
    print(json.dumps({'deployment':result['deployment'],'known_test':models[result['deployment']]['test'],'triage_test':triage['test'],'unseen_style_review':result['unseen_style_triage'],'intervals':intervals},indent=2),flush=True)


if __name__=='__main__':main()
```

scripts/export_encoders.py

```
"""Rebuild frozen feature graphs into a separate directory; preserve supplied models."""
from pathlib import Path
import argparse,hashlib,json,os
import numpy as np,onnx,onnx.utils,onnxruntime as ort,requests,torch
from torchvision.models.video import mc3_18,MC3_18_Weights
ROOT=Path(__file__).resolve().parents[1]
os.environ.setdefault('TORCH_HOME',str(ROOT/'.build/torch_cache'))
FRAME_URL='https://media.githubusercontent.com/media/onnx/models/main/validated/vision/classification/mobilenet/model/mobilenetv2-12.onnx'
FRAME_SOURCE_SHA='c0c3f76d93fa3fd6580652a45618618a220fced18babf65774ed169de0432ad5'


def sha(path):return hashlib.sha256(path.read_bytes()).hexdigest()


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--output-dir',default='.build/exported_encoders');args=parser.parse_args();out=ROOT/args.output_dir
    if out.resolve()==(ROOT/'models').resolve():raise ValueError('Use a separate export directory; do not replace benchmark encoders.')
    out.mkdir(parents=True,exist_ok=True);torch.set_num_threads(4);torch.manual_seed(604)
    backbone=mc3_18(weights=MC3_18_Weights.KINETICS400_V1).eval()
    class TemporalEncoder(torch.nn.Module):
        def __init__(self):
            super().__init__();self.features=torch.nn.Sequential(backbone.stem,backbone.layer1,backbone.layer2,backbone.layer3,backbone.layer4)
        def forward(self,x):return self.features(x).mean(dim=(-1,-2)).transpose(1,2)
    model=TemporalEncoder().eval();batch=torch.randn(1,3,24,112,112);video_path=out/'mc3_features.onnx'
    torch.onnx.export(model,batch,str(video_path),input_names=['video'],output_names=['features'],opset_version=17,dynamo=False)
    opts=ort.SessionOptions();opts.intra_op_num_threads=4;opts.inter_op_num_threads=1
    session=ort.InferenceSession(str(video_path),sess_options=opts,providers=['CPUExecutionProvider'])
    with torch.inference_mode():expected=model(batch).numpy()
    actual=session.run(None,{'video':batch.numpy()})[0];error=float(np.max(np.abs(expected-actual)))
```

scripts/export_encoders.py

```
    np.testing.assert_allclose(actual,expected,atol=2e-4,rtol=2e-4)
    cache=ROOT/'.build/encoder_sources';cache.mkdir(parents=True,exist_ok=True);frame_source=cache/'mobilenetv2-12.onnx'
    if not frame_source.exists():
        response=requests.get(FRAME_URL,timeout=120);response.raise_for_status();frame_source.write_bytes(response.content)
    if sha(frame_source)!=FRAME_SOURCE_SHA:raise ValueError('Frame-encoder source checksum changed.')
    graph=onnx.load(frame_source);pools=[n for n in graph.graph.node if n.op_type=='GlobalAveragePool']
    if len(pools)!=1:raise ValueError('Expected one global pooling node.')
    frame_path=out/'mobilenet_features.onnx';onnx.utils.extract_model(str(frame_source),str(frame_path),[graph.graph.input[0].name],[pools[0].output[0]])
    onnx.checker.check_model(onnx.load(frame_path))
    record={'video_source':'https://download.pytorch.org/models/mc3_18-a90a0ba3.pth','video_sha256':sha(video_path),'video_max_abs_export_error':error,'frame_source':FRAME_URL,'frame_source_sha256':FRAME_SOURCE_SHA,'frame_sha256':sha(frame_path),'note':'These exports are separate from the supplied benchmark graphs. Reaudit/reextract before substituting a graph with different bytes.'}
    (out/'export_check.json').write_text(json.dumps(record,indent=2));print(json.dumps(record,indent=2))


if __name__=='__main__':main()
```

scripts/export_model.py

```
"""Export the development-selected model without accessing test predictions."""
from pathlib import Path
import argparse,json,shutil
import numpy as np
from study import ROOT,sha
from cricketshot.policy import CLASSES


def export(out,deploy=False):
    out=Path(out);selection=json.loads((out/'selection.json').read_text());policy=json.loads((out/'policy.json').read_text())
    assert selection['scope_fitted'] and selection['deployment']==policy['deployment']
    models=out/'deployment';models.mkdir(exist_ok=True);members=selection['candidates'][selection['deployment']]['members'];bundle={}
    for name in members:
        with np.load(out/(name+'.npz'),allow_pickle=False) as z:bundle.update({name+'__'+k:z[k].copy() for k in z.files})
    np.savez_compressed(models/'heads.npz',**bundle)
    with np.load(out/'scope_head.npz',allow_pickle=False) as z:np.savez_compressed(models/'scope_head.npz',**{k:z[k].copy() for k in z.files})
    baseline_names=[n for n in selection['candidates'] if selection['heads'].get(n,{}).get('representation')=='video_mean'];baseline=max(baseline_names,key=lambda n:selection['candidates'][n]['tune']['macro_f1'])
    shutil.copy2(out/(baseline+'.npz'),models/'baseline.npz')
    metadata={'schema_version':4,'classes':CLASSES,'name':'Six-family selective recognizer','deployment':selection['deployment'],'members':[dict(name=n,**selection['heads'][n]) for n in members],'threshold':policy['threshold'],'calibration':policy['calibration'],'baseline':baseline,'baseline_sha256':sha(models/'baseline.npz'),'heads_sha256':sha(models/'heads.npz'),'scope_sha256':sha(models/'scope_head.npz'),'protocol_sha256':selection['protocol_sha256'],'manifest_sha256':selection['manifest_sha256'],'decision_source_sha256':policy['policy_code_sha256'],'feature_source_sha256':sha(ROOT/'src/cricketshot/features.py')}
    metadata['scope_representation']=selection.get('scope_representation','context_mean')
    metadata['scope_followup']=selection.get('scope_followup')
    (models/'classifier.json').write_text(json.dumps(metadata,indent=2))
    if deploy:
        for name in ['heads.npz','scope_head.npz','baseline.npz','classifier.json']:shutil.copy2(models/name,ROOT/'models'/name)
    return metadata


def main():
```

scripts/export_model.py

```
    parser=argparse.ArgumentParser();parser.add_argument('--output-dir',default='results');parser.add_argument('--deploy',action='store_true');args=parser.parse_args();m=export(ROOT/args.output_dir,args.deploy)

print(json.dumps({'deployment':m['deployment'],'threshold':m['threshold'],'installed':args.deploy},indent=2))


if __name__=='__main__':main()
```

scripts/extract_features.py

```
"""Extract frozen representations, without fitting heads or scoring labels."""
from pathlib import Path
from concurrent.futures import ThreadPoolExecutor
import argparse,csv,json,sys,time
import cv2,numpy as np
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT/'src'))
from cricketshot.features import decode,decode_focused,sample_indices,windows,VideoEncoder,FrameEncoder,feature_binding


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--split',nargs='+',choices=['train','tune','calibration','policy','test'],default=['train','tune','calibration','policy']);args=parser.parse_args()
    cv2.setNumThreads(1);rows=[r for r in csv.DictReader((ROOT/'data/manifest.csv').open()) if r['split'] in args.split]
    rows.sort(key=lambda r:(['train','tune','calibration','policy','test'].index(r['split']),r['label'],r['clip_id']))
    cache=ROOT/'.build/features';cache.mkdir(parents=True,exist_ok=True);binding=feature_binding(ROOT);marker=cache/'binding.json'
    if marker.exists():assert json.loads(marker.read_text())['sha256']==binding,'Feature cache binding changed.'
    marker.write_text(json.dumps({'sha256':binding}))
    encoder=VideoEncoder(ROOT/'models/mc3_features.onnx');frame_encoder=FrameEncoder(ROOT/'models/mobilenet_features.onnx')
    started=time.perf_counter();new=0
    def load(r):
        path=cache/(r['label']+'_'+r['clip_id']+'.npz')
        if path.is_file():
            with np.load(path,allow_pickle=False) as z:
                assert str(z['binding'])==binding and str(z['input_sha256'])==r['sha256']
                assert z['video'].shape==(24,512) and z['focused'].shape==(24,1280)
            return r,path,None,None
        full,_=decode(ROOT/r['path']);focused=decode_focused(ROOT/r['path']);assert len(full)==len(focused)
        ids=sample_indices(len(full));return r,path,full[ids],focused[ids]
    with ThreadPoolExecutor(max_workers=2) as pool:
        pending=[]
        for i,r in enumerate(rows):
            pending.append(pool.submit(load,r))
            if len(pending)>=3 or i==len(rows)-1:
                while pending and (len(pending)>=3 or i==len(rows)-1):
                    row,path,full,focused=pending.pop(0).result()
                    if full is not None:

indices,motion=windows(full);video=encoder.encode(full);frames=frame_encoder.encode(focused)
                        temp=path.with_suffix('.tmp.npz');np.savez_compressed(temp,video=video,focused=frames,indices=json.dumps({k:v.tolist() for k,v in indices.items()}),motion=json.dumps(motion),input_sha256=row['sha256'],binding=binding);temp.replace(path);new+=1
                    done=i-len(pending)+1
                    if done%100==0 or done==len(rows):print(f'Features {done}/{len(rows)}, newly extracted {new}, elapsed {(time.perf_counter()-started)/60:.1f} min',flush=True)
    print('Finished frozen-feature extraction',flush=True)


if __name__=='__main__':main()
```

scripts/fetch_data.py

scripts/fetch_data.py

```
"""Download pinned author archives for a reproducible non-commercial research study.

No downloaded code is executed. Original footage rights remain upstream; cite
CricShot10k. Inference already includes its small labelled demonstration set.
"""
from pathlib import Path,PurePosixPath
from concurrent.futures import ThreadPoolExecutor
import argparse,hashlib,json,shutil,subprocess
import gdown
ROOT=Path(__file__).resolve().parents[1]


def sha(path):
    with path.open('rb') as f:return hashlib.file_digest(f,'sha256').hexdigest()


def fetch(record):
    label=record['label'];base=ROOT/'data';archive=base/'downloads'/(label+'.rar');archive.parent.mkdir(parents=True,exist_ok=True)
    if not archive.is_file():
        result=gdown.download(id=record['file_id'],output=str(archive)+'.part',quiet=True,resume=True,use_cookies=False)
        if not result:raise RuntimeError('Archive unavailable: '+label)
        Path(result).replace(archive)
    if sha(archive)!=record['sha256']:raise ValueError('Author archive changed: '+label+'. Use a separately audited experiment.')
    target=base/'raw'/label;target.mkdir(parents=True,exist_ok=True);marker=target/'.extracted.json'
    if marker.is_file():
        assert json.loads(marker.read_text())['archive_sha256']==record['sha256'];return label
    tar=shutil.which('bsdtar') or shutil.which('tar')
    if not tar:raise RuntimeError('Install libarchive/bsdtar for RAR extraction. Windows includes tar.')
    names=subprocess.check_output([tar,'-tf',str(archive)],text=True).splitlines()
    for name in names:
        normalized=name.replace('\\','/');path=PurePosixPath(normalized)
        if path.is_absolute() or '..' in path.parts or ':' in normalized:raise ValueError('Unsafe archive member')
    verbose=subprocess.check_output([tar,'-tvf',str(archive)],text=True)
    if any(line.startswith(('l','h')) for line in verbose.splitlines()):raise ValueError('Archive links are not accepted.')
    subprocess.run([tar,'-xf',str(archive),'-C',str(target)],check=True)
    marker.write_text(json.dumps({'archive_sha256':record['sha256']}));print('Fetched',label,flush=True);return label


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--workers',type=int,default=3);args=parser.parse_args()
    provenance=json.loads((ROOT/'data/provenance.json').read_text())
    if sha(ROOT/'data/source_videos.csv')!=provenance['source_table_sha256']:raise ValueError('Source table snapshot changed.')
    with ThreadPoolExecutor(max_workers=args.workers) as pool:list(pool.map(fetch,provenance['archives']))
    print('All 15 archives match the recorded snapshot.',flush=True)


if __name__=='__main__':main()
```

scripts/prepare_data.py

scripts/prepare_data.py

```
"""Audit all native classes before freezing six-family development/evaluation roles."""
from pathlib import Path
from collections import Counter,defaultdict
from concurrent.futures import ThreadPoolExecutor
import csv,hashlib,json,re
import cv2,imagehash,numpy as np
from PIL import Image
from sklearn.model_selection import StratifiedGroupKFold
ROOT=Path(__file__).resolve().parents[1]


def sha(path):
    with Path(path).open('rb') as f:return hashlib.file_digest(f,'sha256').hexdigest()


class Groups:
    def __init__(self,ids):self.p={i:i for i in ids}
    def root(self,i):
        while self.p[i]!=i:self.p[i]=self.p[self.p[i]];i=self.p[i]
        return i
    def join(self,a,b):
        a,b=self.root(a),self.root(b);self.p[max(a,b)]=min(a,b)


def inspect(path):
    label=path.relative_to(ROOT/'data/raw').parts[0];match=re.fullmatch(r'vid(\d+)_(\d+)',path.stem,re.I)
    if not match:raise ValueError('Unrecognised source identity: '+str(path))
    cap=cv2.VideoCapture(str(path));fps=cap.get(cv2.CAP_PROP_FPS);frames=[]
    try:
        while True:
            ok,bgr=cap.read()
            if not ok:break
            frames.append(cv2.resize(cv2.cvtColor(bgr,cv2.COLOR_BGR2RGB),(96,54),interpolation=cv2.INTER_AREA))
            if len(frames)>1500:raise ValueError('Unexpectedly long dataset clip')
    finally:cap.release()
    if not frames or fps<=0:raise ValueError('Unreadable clip: '+str(path))
    sampled=np.stack([frames[i] for i in np.rint(np.linspace(0,len(frames)-1,16)).astype(int)])
    thumbs=sampled[[0,8,15]]
    return {'path':path.relative_to(ROOT).as_posix(),'label':label,'clip_id':path.stem,'source_id':int(match[1])
,'sha256':sha(path),'decoded_sha256':hashlib.sha256(sampled.tobytes()).hexdigest(),'frames':len(frames),'fps':fp
s,'duration':len(frames)/fps,'phash':[str(imagehash.phash(Image.fromarray(t))) for t in thumbs]},thumbs


def main():
    cv2.setNumThreads(1);cfg=json.loads((ROOT/'configs/experiment.json').read_text());out=ROOT/'data'
    files=sorted(p for p in (out/'raw').rglob('*') if p.suffix.lower() in {'.avi','.mp4','.mov','.mkv'})
    expected={r['label']:r['video_files'] for r in json.loads((out/'provenance.json').read_text())['archives']}
    actual=dict(Counter(p.relative_to(out/'raw').parts[0] for p in files))
    if actual!=expected:raise ValueError('The raw archive inventory is incomplete or changed. Run fetch_data.py
and check the pinned counts before preparing a split.')
    inspected=[]
    with ThreadPoolExecutor(max_workers=6) as pool:
        for i,item in enumerate(pool.map(inspect,files),1):
            inspected.append(item)
            if i%500==0:print('Audited',i,'/',len(files),flush=True)
    rows=[r for r,t in inspected];thumbs=np.stack([t for r,t in inspected]);ids={r['source_id'] for r in
rows};groups=Groups(ids)
    previous={int(r['source_id']):r for r in csv.DictReader((out/'previous_sources.csv').open())}
    inherited=defaultdict(list)
    for sid,r in previous.items():
        if sid in ids:inherited[r['previous_group']].append(sid)
```

scripts/prepare_data.py

```
    for members in inherited.values():
        for sid in members[1:]:groups.join(members[0],sid)
    aliases=defaultdict(list);alias_edges=[]
    for r in csv.DictReader((out/'source_videos.csv').open(encoding='utf-8-sig')):
        sid=int(r['Video No'])
        if sid not in ids:continue
        detail=re.sub(r'\|?\s*day\s*\d+','',r['Details (series, match no, year)'],flags=re.I)
        key=re.sub(r'\W+','',(r['Team vs Team']+detail).casefold());aliases[key].append(sid)
    for members in aliases.values():
        for sid in members[1:]:groups.join(members[0],sid);alias_edges.append([members[0],sid])
    for r in rows:r['target']=cfg['source_mapping'].get(r['label'],'review')
    seen={};duplicate={};conflicts=set();edges=[]
    def link(i,j,kind,mae=None):
        groups.join(rows[i]['source_id'],rows[j]['source_id']);duplicate.setdefault(j,i)
        edges.append({'a':i,'b':j,'type':kind,**({'mae_255':mae} if mae is not None else {})})
        if rows[i]['target']!=rows[j]['target']:conflicts.update((i,j))
    for i,r in enumerate(rows):
        for key in [('file',r['sha256']),('decoded',r['decoded_sha256'])]:
            if key in seen:link(seen[key],i,key[0])
            else:seen[key]=i
    hashes=np.array([[int(h,16) for h in r['phash']] for r in rows],dtype=np.uint64)
    lut=np.array([i.bit_count() for i in range(256)],dtype=np.uint8);durations=np.array([r['duration'] for r in
rows])
    for i,r in enumerate(rows):
        distances=lut[np.bitwise_xor(hashes[i+1:],hashes[i]).view(np.uint8).reshape(-1,3,8)].sum(axis=2)
        candidates=np.flatnonzero((distances.sum(axis=1)<=8)&(distances.max(axis=1)<=4))+i+1
        for j in candidates:
            if abs(durations[i]-durations[j])/max(durations[i],durations[j])>.15:continue
            mae=float(np.abs(thumbs[i].astype(np.int16)-thumbs[j].astype(np.int16)).mean())
            if mae<=5:link(i,int(j),'near',mae)
    component_roles=defaultdict(set)
    for sid,r in previous.items():
        if sid in ids:component_roles[groups.root(sid)].add(r['previous_split'])
    bridged={g for g,roles in component_roles.items() if len(roles)>1}
    val=[]
    for i,r in enumerate(rows):
        r['group_id']=groups.root(r['source_id']);roles=component_roles[r['group_id']]
        r['excluded_reason']=('compilation_source' if r['source_id'] in cfg['excluded_source_ids'] else
'partition_bridge' if r['group_id'] in bridged else 'conflicting_target_duplicate' if i in conflicts else
'duplicate_clip' if i in duplicate else '')
        if r['excluded_reason']:r['split']='excluded'
        elif roles=={'train'}:r['split']='train'
        elif roles=={'validation'}:r['split']='validation';val.append(r)
        else:r['split']='test'
    assign=[r for r in val if r['label'] not in cfg['unseen_style_classes']]
    y=np.array([cfg['classes'].index(r['target']) if r['target']!='review' else 6 for r in
assign]);g=np.array([r['group_id'] for r in assign]);role={}
    cv=StratifiedGroupKFold(n_splits=3,shuffle=True,random_state=cfg['seed'])
    for fold,(_,idx) in enumerate(cv.split(np.zeros(len(y)),y,g)):
        for i in idx:role[assign[i]['group_id']]=['tune','calibration','policy'][fold]
    for r in val:r['split']=role.get(r['group_id'],'reserved_unknown')
    for r in rows:
        if r['label'] in cfg['unseen_style_classes'] and r['split'] not in
{'test','excluded'}:r['split']='reserved_unknown'
    fields=list(rows[0])
    with (out/'manifest.csv').open('w',newline='',encoding='utf-8') as f:
        w=csv.DictWriter(f,fieldnames=fields);w.writeheader();w.writerows({**r,'phash':json.dumps(r['phash'])}
for r in rows)
    splits=['train','tune','calibration','policy','test','reserved_unknown','excluded']
    counts={s:dict(Counter(r['target'] for r in rows if r['split']==s)) for s in splits}
    role_groups={s:{r['group_id'] for r in rows if r['split']==s} for s in splits[:5]}
    for a in role_groups:
        for b in role_groups:
            if a!=b:assert not role_groups[a]&role_groups[b],(a,b)
```

scripts/prepare_data.py

```
    for s in splits[:5]:assert set(cfg['classes']).issubset(counts[s]),(s,counts[s])
    audit={'raw_clips':len(rows),'retained_clips':sum(r['split']!='excluded' for r in rows),'raw_class_counts':dict(Counter(r['label'] for r in rows)),'counts':counts,'native_counts':{s:dict(Counter(r['label'] for r in rows if r['split']==s)) for s in splits},'group_counts':{s:len(g) for s,g in role_groups.items()},'source_ids':len(ids),'bridged_components':sorted(bridged),'exclusions':dict(Counter(r['excluded_reason'] for r in rows if r['excluded_reason'])),'metadata_alias_edges':alias_edges,'duplicate_edges':edges,'protocol_sha256':sha(ROOT/'configs/experiment.json'),'manifest_sha256':sha(out/'manifest.csv'),'test_limit':'Reused old benchmark groups plus any newly available source components. Source groups are not independently verified matches.'}
    (out/'audit.json').write_text(json.dumps(audit,indent=2));print(json.dumps({k:v for k,v in audit.items() if k not in {'duplicate_edges','metadata_alias_edges'}},indent=2),flush=True)


if __name__=='__main__':main()
```

scripts/refine_classes.py

```
"""Development-only dimension follow-up; preserve all earlier fitted candidates."""
import json,time
from train import neural
from study import ROOT,read,sha,data,fit_transform,candidate_probabilities,metrics


def main():
    out=ROOT/'results';cfg=read('configs/class_followup.json');base=read('configs/experiment.json');selection=read('results/selection.json')
    if (out/'metrics.json').exists() or (out/'policy.json').exists() or selection.get('class_followup'):raise RuntimeError('A new study is required after a frozen policy or test.')
    rows,x,y,splits,groups=data();train=(splits=='train')&(y>=0);tune=(splits=='tune')&(y>=0);trace=read('results/training_trace.json');ensembles={}
    original=selection['deployment']
    for rep in cfg['representations']:
        state,z=fit_transform(x[rep],train,cfg['pca'],base['seed']);members=[]
        for seed in cfg['seeds']:
            name=f'{rep}_pca{cfg["pca"]}_h{cfg["hidden"]}_seed{seed}'
            record,history=neural(z,y,splits,state,base['neural'],cfg['hidden'],seed,name,out);trace.extend(history);selection['heads'][name]={**record,'representation':rep,'pca':cfg['pca']};members.append(name)

name=f'{rep}_pca{cfg["pca"]}_ensemble';selection['ensembles'][name]={'members':members};ensembles[rep]=members

name=f'full_motion_pca{cfg["pca"]}_ensemble';selection['ensembles'][name]={'members':sum(ensembles.values(),[])}
    p=candidate_probabilities(selection,x,out)
    for name in [f'{r}_pca{cfg["pca"]}_ensemble' for r in cfg['representations']]+[name]:
        selection['candidates'][name]={'members':selection['ensembles'][name]['members'],'train':metrics(y[train],p[name][train].argmax(axis=1)),'tune':metrics(y[tune],p[name][tune].argmax(axis=1))}
    selection['deployment']=max(selection['candidates'],key=lambda n:selection['candidates'][n]['tune']['macro_f1'])
    selection['class_followup']={'protocol_sha256':sha(ROOT/'configs/class_followup.json'),'training_code_sha256':sha(ROOT/'scripts/refine_classes.py'),'previous_deployment':original,'test_accessed':False}
    selection['finalized_utc']=time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime());(out/'selection.json').write_text(json.dumps(selection,indent=2));(out/'training_trace.json').write_text(json.dumps(trace,indent=2))
    chosen=selection['candidates'][selection['deployment']];print(json.dumps({'selected':selection['deployment'],'train':chosen['train']['accuracy'],'tune':chosen['tune']['accuracy'],'tune_f1':chosen['tune']['macro_f1']},indent=2),flush=True)


if __name__=='__main__':main()
```

scripts/refine_scope.py

scripts/refine_scope.py

```
"""Recorded development follow-up after the linear scope coverage shortfall."""
import json,shutil,time
import numpy as np,torch
from sklearn.metrics import roc_auc_score
from threadpoolctl import threadpool_limits
from study import ROOT,read,sha,data,fit_transform
from cricketshot.policy import head_logits


def main():
    out=ROOT/'results';cfg=read('configs/scope_experiment.json');selection=read('results/selection.json')
    if (out/'metrics.json').exists() or selection.get('scope_followup'):raise RuntimeError('Follow-up already fitted or test opened.')
    baseline=out/'scope_baseline';baseline.mkdir(exist_ok=False)
    for name in ['selection.json','scope_head.npz','calibration.json','policy.json']:shutil.copy2(out/name,baseline/name)
    rows,x,y,splits,groups=data();train=splits=='train';tune=splits=='tune';target=(y>=0).astype(np.float32)
    torch.set_num_threads(4);torch.use_deterministic_algorithms(True)
    best=max(selection['scope_grid'],key=lambda r:r['tune_auroc'])['tune_auroc'];chosen={'name':'linear_baseline','representation':'context_mean','head_type':'linear','C':selection['scope_C'],'tune_auroc':best}
    with np.load(baseline/'scope_head.npz',allow_pickle=False) as z:best_head={k:z[k].copy() for k in z.files}
    records=[chosen.copy()];traces=[]
    for spec in cfg['representations']:
        state,z=fit_transform(x[spec['name']],train,spec['pca']);tx=torch.from_numpy(z[train]);vx=torch.from_numpy(z[tune]);ty=torch.from_numpy(target[train,None])
        positive=float((ty==1).sum());negative=len(ty)-positive;positive_weight=negative/positive
        for seed in cfg['seeds']:
            torch.manual_seed(seed);np.random.seed(seed);name=f"{spec['name']}_pca{spec['pca']}_h{spec['hidden']}_scope_seed{seed}"
            model=torch.nn.Sequential(torch.nn.Linear(spec['pca'],spec['hidden']),torch.nn.ReLU(),torch.nn.Dropout(cfg['dropout']),torch.nn.Linear(spec['hidden'],1))
            optimizer=torch.optim.AdamW(model.parameters(),lr=cfg['learning_rate'],weight_decay=cfg['weight_decay']);lossfn=torch.nn.BCEWithLogitsLoss(pos_weight=torch.tensor(positive_weight))
            local_best=-1.;stale=0;best_epoch=None;local_head=None
            for epoch in range(1,cfg['max_epochs']+1):
                model.train();loss_sum=0.
                for idx in torch.randperm(len(tx)).split(cfg['batch_size']):
                    optimizer.zero_grad(set_to_none=True);labels=ty[idx]*(1-cfg['label_smoothing'])+cfg['label_smoothing']/2;loss=lossfn(model(tx[idx]+torch.randn_like(tx[idx])*cfg['feature_noise_std']),labels);loss.backward();optimizer.step();loss_sum+=float(loss.detach())*len(idx)
                model.eval()
                with torch.inference_mode():tr=model(tx).numpy().ravel();va=model(vx).numpy().ravel()
                auc=float(roc_auc_score(target[tune],va));tr_auc=float(roc_auc_score(target[train],tr));traces.append({'head':name,'epoch':epoch,'training_loss':loss_sum/len(tx),'train_auroc':tr_auc,'tune_auroc':auc})
                if auc>local_best+1e-12:
                    local_best=auc;stale=0;best_epoch=epoch;local_head={**state,'weight1':model[0].weight.detach().numpy().copy(),'bias1':model[0].bias.detach().numpy().copy(),'weight':model[3].weight.detach().numpy().copy(),'bias':model[3].bias.detach().numpy().copy()}
                else:stale+=1
                if stale>=cfg['patience']:break
            np.savez_compressed(out/(name+'.npz'),**local_head)
            record={'name':name,'representation':spec['name'],'pca':spec['pca'],'hidden':spec['hidden'],'seed':seed,'head_type':'mlp','best_epoch':best_epoch,'tune_auroc':local_best};records.append(record)
            if local_best>best+1e-12:best=local_best;best_head=local_head;chosen=record
            print(name,'epoch',best_epoch,'tune AUROC',round(local_best,4),flush=True)
    np.savez_compressed(out/'scope_head.npz',**best_head)
    selection['scope_followup']={'protocol_sha256':sha(ROOT/'configs/scope_experiment.json'),'training_code_sha256':sha(ROOT/'scripts/refine_scope.py'),'reason':cfg['reason'],'candidates':records,'selected':chosen,'test_accessed':False}
    selection['scope_representation']=chosen['representation'];selection['finalized_utc']=time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime())
```

scripts/refine_scope.py

```
    (out/'selection.json').write_text(json.dumps(selection,indent=2));(out/'scope_trace.json').write_text(json.dumps(traces,indent=2))
    for name in ['calibration.json','policy.json']:(out/name).unlink()
    print(json.dumps({'chosen_scope':chosen,'baseline_preserved':'results/scope_baseline'},indent=2),flush=True)


if __name__=='__main__':main()
```

scripts/select_operating_point.py

```
"""Freeze the requested coverage-oriented default before test evaluation."""
import json,time
import numpy as np
from threadpoolctl import threadpool_limits
from study import ROOT,read,sha,data,candidate_probabilities,selective
from cricketshot.policy import head_logits,calibrated_support


def main():
    out=ROOT/'results';cfg=read('configs/operating_point.json');fixed=read('results/policy.json');selection=read('results/selection.json')
    if (out/'metrics.json').exists() or fixed.get('operating_point'):raise RuntimeError('The operating point is already frozen or evaluated.')
    (out/'cautious_policy.json').write_text(json.dumps(fixed,indent=2))
    chosen=next(r for r in fixed['grid'] if r['threshold']==cfg['threshold'])
    fixed.update(threshold=cfg['threshold'],policy_metrics=chosen,accuracy_and_rejection_targets_met=chosen['eligible'],known_coverage_goal_met=chosen['known_coverage']>=fixed['targets']['known_coverage_goal'])
    fixed['operating_point']={**cfg,'config_sha256':sha(ROOT/'configs/operating_point.json'),'code_sha256':sha(ROOT/'scripts/select_operating_point.py')}
    g=cfg['development_goals'];fixed['operating_point']['development_goals_met']=bool(chosen['known_coverage']>=g['known_coverage'] and chosen['known_automatic_accuracy']>=g['known_automatic_accuracy'] and chosen['out_of_scope_review_rate']>=g['exposure_review_rate'])
    rows,x,y,splits,groups=data();p=candidate_probabilities(selection,x,out)[selection['deployment']]
    with np.load(out/'scope_head.npz',allow_pickle=False) as z:head={k:z[k].copy() for k in z.files}
    with threadpool_limits(limits=4):logits=head_logits(x[selection.get('scope_representation','context_mean')],head).ravel()
    support,conditional,scope=calibrated_support(p,logits,fixed['calibration']);mask=splits=='policy'
    fixed['confidence_only_same_threshold']=selective(y[mask],conditional[mask],fixed['threshold'])
    grid=[dict(selective(y[mask],conditional[mask],t),threshold=t) for t in read('configs/experiment.json')['policy_grid']]
    matched=min(grid,key=lambda r:(abs(r['known_coverage']-chosen['known_coverage']),-(r['known_automatic_accuracy'] or 0),r['threshold']))
    fixed['confidence_only_matched_coverage']={'threshold':matched['threshold'],'policy_metrics':matched,'selection':'Closest known-shot coverage to the main policy on policy groups only; ties prefer higher automatic known accuracy, then lower threshold.'}
    fixed['finalized_utc']=time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime());(out/'policy.json').write_text(json.dumps(fixed,indent=2))
    print(json.dumps({'threshold':fixed['threshold'],'metrics':chosen,'development_goals_met':fixed['operating_point']['development_goals_met']},indent=2))


if __name__=='__main__':main()
```

scripts/study.py

```
"""Shared study loading, train-only projections and transparent evaluation arithmetic."""
from pathlib import Path
import csv,hashlib,json,sys
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits
```

scripts/study.py

```
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT/'src'))
from cricketshot.features import describe,feature_binding
from cricketshot.policy import CLASSES,head_logits,softmax,calibrated_support


def read(path):return json.loads((ROOT/path).read_text())
def sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def data(include_test=False,known_only=False):
    binding=feature_binding(ROOT)
    rows=[r for r in csv.DictReader((ROOT/'data/manifest.csv').open()) if r['split'] in (['train','tune','calibration','policy','test'] if include_test else ['train','tune','calibration','policy']) and (not known_only or r['target']!='review')]
    modes=read('configs/experiment.json')['representations']
    # This local cache contains frozen descriptors only, never predictions or fits.
    # Bind it to the exact audited row order and descriptor implementation.
    cache=ROOT/'.build/study_cache'/('all' if include_test else 'development')/('known' if known_only else 'mixed')
    signature=hashlib.sha256(json.dumps([binding,modes,[(r['path'],r['sha256'],r['split']) for r in rows]]).encode()).hexdigest()
    marker=cache/'binding.json';arrays=cache/'descriptors.npz'
    if marker.exists() and arrays.exists():
        recorded=json.loads(marker.read_text())
        if recorded['binding']==signature and recorded['sha256']==sha(arrays):
            with np.load(arrays,allow_pickle=False) as z:
                x={m:z[m].copy() for m in modes}
                return rows,x,z['y'].copy(),z['splits'].copy(),z['groups'].copy()
    values={m:[] for m in modes}
    for r in rows:
        path=ROOT/'.build/features'/(r['label']+'_'+r['clip_id']+'.npz')
        with np.load(path,allow_pickle=False) as z:
            assert str(z['binding'])==binding and str(z['input_sha256'])==r['sha256']
            indices={k:np.asarray(v) for k,v in json.loads(str(z['indices'])).items()}
            for m in modes:values[m].append(describe(z['video'],z['focused'],indices,m))
    x={m:np.stack(v) for m,v in values.items()}
    y=np.array([CLASSES.index(r['target']) if r['target']!='review' else -1 for r in rows]);splits=np.array([r['split'] for r in rows]);groups=np.array([int(r['group_id']) for r in rows])
    cache.mkdir(parents=True,exist_ok=True)
    np.savez(arrays,**x,y=y,splits=splits,groups=groups)
    marker.write_text(json.dumps({'binding':signature,'sha256':sha(arrays),'content':'Frozen descriptors only; no model outcomes.'}))
    return rows,x,y,splits,groups


def metrics(y,p):
    y=np.asarray(y);p=np.asarray(p);cm=np.bincount(y*6+p,minlength=36).reshape(6,6)
    tp=np.diag(cm);n=cm.sum(axis=1);pred=cm.sum(axis=0)
    precision=np.divide(tp,pred,out=np.zeros(6,float),where=pred>0);recall=np.divide(tp,n,out=np.zeros(6,float),where=n>0)
    f1=np.divide(2*precision*recall,precision+recall,out=np.zeros(6,float),where=(precision+recall)>0)
    return {'n':int(len(y)),'accuracy':float(tp.sum()/len(y)) if len(y) else None,'macro_f1':float(f1.mean()),'balanced_accuracy':float(recall.mean()),'confusion_matrix':cm.tolist(),'per_class':{c:{'precision':float(precision[i]),'recall':float(recall[i]),'f1':float(f1[i]),'support':int(n[i])} for i,c in enumerate(CLASSES)}}


def fit_transform(x,mask,components,seed=604):
    scaler=StandardScaler().fit(x[mask]);mean=scaler.mean_.astype(np.float32);scale=scaler.scale_.astype(np.float32)
    z=np.clip((x-mean)/scale,-5,5).astype(np.float32);state={'mean':mean,'scale':scale,'input_clip':np.array(5,np.float32)}
    if components:
        pca=PCA(n_components=components,whiten=True,svd_solver='randomized',random_state=seed)
        with threadpool_limits(limits=4):pca.fit(z[mask])
```

scripts/study.py

```
        state.update(projection_center=pca.mean_.astype(np.float32),projection=(pca.components_/np.sqrt(np.maximum(pca.explained_variance_[:,None],1e-12))).astype(np.float32))
        z=(z-state['projection_center'])@state['projection'].T
    return state,z.astype(np.float32)


def candidate_probabilities(selection,x,out=ROOT/'results'):
    values={}
    for name,r in selection['heads'].items():
        with np.load(out/(name+'.npz'),allow_pickle=False) as z:h={k:z[k].copy() for k in z.files}
        with threadpool_limits(limits=4):values[name]=softmax(head_logits(x[r['representation']],h))
    for name,r in selection['ensembles'].items():values[name]=np.mean([values[h] for h in r['members']],axis=0)
    return values


def selective(y,support,threshold):
    above=support>threshold;accepted=above.sum(axis=1)==1;pred=support.argmax(axis=1);known=y>=0;unknown=~known
    correct=accepted&known&(pred==y)
    nknown=int(known.sum());nunknown=int(unknown.sum());auto_known=int((accepted&known).sum());auto_all=int(accepted.sum())
    return {'n_known':nknown,'n_out_of_scope':nunknown,'automatic_known':auto_known,'automatic_all':auto_all,'correct_automatic':int(correct.sum()),'known_coverage':float(auto_known/nknown) if nknown else None,'known_automatic_accuracy':float(correct.sum()/auto_known) if auto_known else None,'automatic_accuracy_all':float(correct.sum()/auto_all) if auto_all else None,'out_of_scope_review_rate':float((~accepted&unknown).sum()/nunknown) if nunknown else None,'out_of_scope_false_automatic':int((accepted&unknown).sum()),'zero_candidates':int((above.sum(axis=1)==0).sum()),'multiple_candidates':int((above.sum(axis=1)>1).sum()),'per_class':{c:{'n':int((y==i).sum()),'automatic':int((accepted&(y==i)).sum()),'coverage':float((accepted&(y==i)).sum()/(y==i).sum()) if (y==i).any() else None,'automatic_accuracy':float(correct[y==i].sum()/(accepted&(y==i)).sum()) if (accepted&(y==i)).any() else None} for i,c in enumerate(CLASSES)}}


def nll_ece(y,p,bins=15):
    confidence=p.max(axis=1);correct=p.argmax(axis=1)==y;ece=0.
    for a,b in zip(np.linspace(0,1,bins+1)[:-1],np.linspace(0,1,bins+1)[1:]):
        mask=(confidence>=a)&(confidence<b if b<1 else confidence<=b)
        if mask.any():ece+=mask.mean()*abs(correct[mask].mean()-confidence[mask].mean())
    return {'nll':float(-np.log(np.maximum(p[np.arange(len(y)),y],1e-12)).mean()),'ece_15_bins':float(ece),'brier':float(np.square(p-np.eye(6)[y]).sum(axis=1).mean())}
```

scripts/train.py

```
"""Fit six-family heads and a scope head; test rows are never loaded here."""
from pathlib import Path
import argparse,json,time
import numpy as np,torch
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from threadpoolctl import threadpool_limits
from study import ROOT,read,sha,data,metrics,fit_transform,candidate_probabilities
from cricketshot.policy import CLASSES,head_logits,softmax


def neural(x,y,splits,state,hp,hidden,seed,name,out):
    torch.set_num_threads(4);torch.use_deterministic_algorithms(True);torch.manual_seed(seed);np.random.seed(seed)
    train=(splits=='train')&(y>=0);tune=(splits=='tune')&(y>=0);tx=torch.from_numpy(x[train]);ty=torch.from_numpy(y[train]);vx=torch.from_numpy(x[tune])
    model=torch.nn.Sequential(torch.nn.Linear(x.shape[1],hidden),torch.nn.ReLU(),torch.nn.Dropout(hp['dropout']),torch.nn.Linear(hidden,6))
    optimizer=torch.optim.AdamW(model.parameters(),lr=hp['learning_rate'],weight_decay=hp['weight_decay'])
    counts=np.bincount(y[train],minlength=6);weights=len(ty)/(6*counts)
    lossfn=torch.nn.CrossEntropyLoss(weight=torch.tensor(weights,dtype=torch.float32),label_smoothing=hp['label_
```

scripts/train.py

```
    smoothing'])
    best=-1.;best_head=None;best_epoch=None;stale=0;trace=[]
    for epoch in range(1,hp['max_epochs']+1):
        model.train();loss_sum=0.
        for idx in torch.randperm(len(tx)).split(hp['batch_size']):
            optimizer.zero_grad(set_to_none=True);inputs=tx[idx]+torch.randn_like(tx[idx])*hp['feature_noise_std
'];loss=lossfn(model(inputs),ty[idx]);loss.backward();optimizer.step();loss_sum+=float(loss.detach())*len(idx)
        model.eval()
        with torch.inference_mode():
            tr=model(tx).numpy();va=model(vx).numpy()
        tm=metrics(y[train],tr.argmax(axis=1));vm=metrics(y[tune],va.argmax(axis=1))
        trace.append({'head':name,'epoch':epoch,'training_cross_entropy':loss_sum/len(tx),'train_accuracy':tm['a
ccuracy'],'train_macro_f1':tm['macro_f1'],'tune_accuracy':vm['accuracy'],'tune_macro_f1':vm['macro_f1']})
        if vm['macro_f1']>best+1e-12:
            best=vm['macro_f1'];best_epoch=epoch;stale=0;best_head={**state,'weight1':model[0].weight.detach().n
umpy().copy(),'bias1':model[0].bias.detach().numpy().copy(),'weight':model[3].weight.detach().numpy().copy(),'bi
as':model[3].bias.detach().numpy().copy()}
        else:stale+=1
        if stale>=hp['patience']:break
    np.savez_compressed(out/(name+'.npz'),**best_head)
    print('Neural head',name,'epoch',best_epoch,'tune F1',round(best,4),flush=True)
    return {'head_type':'mlp','seed':seed,'best_epoch':best_epoch,'hidden':hidden},trace


def fit_scope(selected,out,cfg):
    if selected.get('scope_fitted'):raise RuntimeError('Scope selection is already frozen.')
    rows,x,y,splits,groups=data();scope_train=splits=='train';scope_tune=splits=='tune';known=(y>=0).astype(int)
    scope_state,scope_x=fit_transform(x['context_mean'],scope_train,128,cfg['seed']);scope_records=[];best_auc=-
1.;scope_head=None
    for c in cfg['linear_C']:
        model=LogisticRegression(C=c,class_weight='balanced',max_iter=3000,tol=1e-5)
        with threadpool_limits(limits=4):model.fit(scope_x[scope_train],known[scope_train]);logits=model.decisio
n_function(scope_x[scope_tune])
        auc=float(roc_auc_score(known[scope_tune],logits));scope_records.append({'C':c,'tune_auroc':auc})
        if auc>best_auc+1e-12:best_auc=auc;scope_head={**scope_state,'weight':model.coef_.astype(np.float32),'bi
as':model.intercept_.astype(np.float32)};selected['scope_C']=c
    np.savez_compressed(out/'scope_head.npz',**scope_head);selected['scope_grid']=scope_records;selected['scope_
fitted']=True
    selected['finalized_utc']=time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime());(out/'selection.json').write_tex
t(json.dumps(selected,indent=2))
    print('Scope tune AUROC',best_auc,flush=True)


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--output-dir',default='results');parser.add_argument('
--phase',choices=['all','classes','scope'],default='all');args=parser.parse_args();out=ROOT/args.output_dir;out.
mkdir(parents=True,exist_ok=True);cfg=read('configs/experiment.json')
    if args.phase=='scope':
        if (out/'policy.json').exists() or (out/'metrics.json').exists():raise RuntimeError('Do not refit after
calibration or test.')
        selected=json.loads((out/'selection.json').read_text());assert
selected['protocol_sha256']==sha(ROOT/'configs/experiment.json');fit_scope(selected,out,cfg);return
    if (out/'selection.json').exists():raise RuntimeError('Use a new output directory rather than overwrite a
frozen fit.')
    rows,x,y,splits,groups=data(known_only=True);train=(splits=='train')&(y>=0);tune=(splits=='tune')&(y>=0)
    selected={'protocol_sha256':sha(ROOT/'configs/experiment.json'),'manifest_sha256':sha(ROOT/'data/manifest.cs
v'),'classes':CLASSES,'heads':{},'ensembles':{},'candidates':{},'test_accessed_during_selection':False,'status':
'selection_frozen','fit_roles':['train'],'selection_role':'tune'};traces=[];transforms={}
    def transform(rep,dim):
        key=(rep,dim)
        if key not in transforms:transforms[key]=fit_transform(x[rep],train,dim,cfg['seed'])
        return transforms[key]
    for rep in cfg['representations']:
        dim=0 if rep=='video_mean' else 128;state,z=transform(rep,dim)
```

scripts/train.py

```
        for c in cfg['linear_C']:
            name=rep+'_linear_C'+str(c).replace('.','p')
            model=LogisticRegression(C=c,class_weight='balanced',max_iter=3000,tol=1e-5)
            with threadpool_limits(limits=4):model.fit(z[train],y[train]);pred=model.predict(z[tune])
            head={**state,'weight':model.coef_.astype(np.float32),'bias':model.intercept_.astype(np.float32)};np.savez_compressed(out/(name+'.npz'),**head)
            with threadpool_limits(limits=4):pred=head_logits(x[rep][tune],head).argmax(axis=1)
            m=metrics(y[tune],pred);selected['heads'][name]={'representation':rep,'head_type':'linear','C':c,'pca':dim};selected['candidates'][name]={'tune':m,'members':[name]}
            print('Linear',name,'tune accuracy',round(m['accuracy'],4),'F1',round(m['macro_f1'],4),flush=True)
    for hp in cfg['neural_configs']:
        ensembles={}
        for rep in cfg['neural_representations']:
            state,z=transform(rep,hp['pca']);members=[]
            for seed in cfg['neural_seeds']:
                name=f'{rep}_pca{hp["pca"]}_h{hp["hidden"]}_seed{seed}'

record,trace=neural(z,y,splits,state,cfg['neural'],hp['hidden'],seed,name,out);traces.extend(trace)
                selected['heads'][name]={**record,'representation':rep,'pca':hp['pca']};members.append(name)

name=f'{rep}_pca{hp["pca"]}_ensemble';selected['ensembles'][name]={'members':members};ensembles[rep]=members

name=f'full_motion_pca{hp["pca"]}_ensemble';selected['ensembles'][name]={'members':sum(ensembles.values(),[])}
    probabilities=candidate_probabilities(selected,x,out)
    for name,r in selected['ensembles'].items():selected['candidates'][name]={'tune':metrics(y[tune],probabilities[name][tune].argmax(axis=1)),'members':r['members']}
    for name,r in selected['candidates'].items():r['train']=metrics(y[train],probabilities[name][train].argmax(axis=1))
    chosen=max(selected['candidates'],key=lambda n:selected['candidates'][n]['tune']['macro_f1']);selected['deployment']=chosen
    selected['scope_fitted']=False;selected['finalized_utc']=time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime());selected['training_code_sha256']=sha(Path(__file__))
    (out/'training_trace.json').write_text(json.dumps(traces,indent=2));(out/'selection.json').write_text(json.dumps(selected,indent=2))
    print(json.dumps({'deployment':chosen,'train':selected['candidates'][chosen]['train']['accuracy'],'tune':selected['candidates'][chosen]['tune']['accuracy'],'tune_macro_f1':selected['candidates'][chosen]['tune']['macro_f1']},indent=2),flush=True)
    if args.phase=='all':fit_scope(selected,out,cfg)


if __name__=='__main__':main()
```

scripts/verify.py

```
"""Independent NumPy evidence checks and fresh inference, using runtime dependencies.

No sklearn/torch or raw dataset download is required. The default replay record
is private; --record explicitly refreshes the submitted runtime evidence.
"""
from pathlib import Path
import argparse,csv,hashlib,json,sys,time
import cv2,numpy as np
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT/'src'))
from cricketshot.policy import CLASSES


def read(p):return json.loads((ROOT/p).read_text())
def sha(p):
    with Path(p).open('rb') as f:return hashlib.file_digest(f,'sha256').hexdigest()


def arithmetic(y,p):
    matrix=np.array([[int(((y==i)&(p==j)).sum()) for j in range(6)] for i in range(6)])
```

scripts/verify.py

```
    stats={};f1=[];recalls=[]
    for i,c in enumerate(CLASSES):
        tp=int(matrix[i,i]);actual=int(matrix[i].sum());predicted=int(matrix[:,i].sum());precision=tp/predicted if predicted else 0.;recall=tp/actual if actual else 0.;f=2*tp/(actual+predicted) if actual+predicted else 0.

stats[c]={'precision':precision,'recall':recall,'f1':f,'support':actual};f1.append(f);recalls.append(recall)
    return {'n':len(y),'accuracy':float((y==p).mean()) if len(y) else None,'macro_f1':float(np.mean(f1)),'balanced_accuracy':float(np.mean(recalls)),'confusion_matrix':matrix.tolist(),'per_class':stats}


def triage(y,support,threshold):
    count=np.sum(support>threshold,axis=1);auto=count==1;pred=np.argmax(support,axis=1);known=y>=0;unknown=y<0;correct=auto&known&(pred==y)
    nk=int(known.sum());nu=int(unknown.sum());ak=int((auto&known).sum());aa=int(auto.sum());nc=int(correct.sum());classes={}
    for i,c in enumerate(CLASSES):
        n=int((y==i).sum());a=int((auto&(y==i)).sum());right=int((correct&(y==i)).sum());classes[c]={'n':n,'automatic':a,'coverage':a/n if n else None,'automatic_accuracy':right/a if a else None}
    return {'n_known':nk,'n_out_of_scope':nu,'automatic_known':ak,'automatic_all':aa,'correct_automatic':nc,'known_coverage':ak/nk if nk else None,'known_automatic_accuracy':nc/ak if ak else None,'automatic_accuracy_all':nc/aa if aa else None,'out_of_scope_review_rate':float((unknown&~auto).sum()/nu) if nu else None,'out_of_scope_false_automatic':int((unknown&auto).sum()),'zero_candidates':int((count==0).sum()),'multiple_candidates':int((count>1).sum()),'per_class':classes}


def quality(y,p):
    confidence=p.max(axis=1);correct=(p.argmax(axis=1)==y).astype(float);ece=0.
    # Equal-width bins, with the final bin including exactly one.
    indices=np.minimum((confidence*15).astype(int),14)
    for b in range(15):
        mask=indices==b
        if mask.any():ece+=mask.sum()/len(y)*abs(float(correct[mask].mean())-float(confidence[mask].mean()))
    truth=np.zeros_like(p);truth[np.arange(len(y)),y]=1
    return {'nll':float(np.mean(-np.log(np.clip(p[np.arange(len(y)),y],1e-12,None)))),'ece_15_bins':float(ece),'brier':float(np.square(p-truth).sum(axis=1).mean())}


CHECKS=0
def same(actual,recorded,path=''):
    global CHECKS
    if isinstance(actual,dict):
        for k,v in actual.items():same(v,recorded[k],path+'/'+k)
    elif isinstance(actual,(list,tuple)):
        assert len(actual)==len(recorded),path
        for i,(a,b) in enumerate(zip(actual,recorded)):same(a,b,path+'/'+str(i))
    elif isinstance(actual,(float,np.floating)):
        assert abs(float(actual)-float(recorded))<1e-10,(path,actual,recorded);CHECKS+=1
    else:assert actual==recorded,(path,actual,recorded);CHECKS+=1


def auc(labels,scores):
    order=np.argsort(scores,kind='stable');sorted_scores=scores[order];ranks=np.empty(len(scores),float);start=0
    while start<len(scores):
        stop=start+1
        while stop<len(scores) and sorted_scores[stop]==sorted_scores[start]:stop+=1
        ranks[order[start:stop]]=(start+1+stop)/2;start=stop
    positives=labels.astype(bool);n=int(positives.sum());m=len(labels)-n
    return float((ranks[positives].sum()-n*(n+1)/2)/(n*m))


def main():
    parser=argparse.ArgumentParser();parser.add_argument('--replay',action='store_true');parser.add_argument('--record',action='store_true');args=parser.parse_args()
    cfg=read('configs/experiment.json');meta=read('models/classifier.json');m=read('results/metrics.json');selec
```

scripts/verify.py

```
tion=read('results/selection.json');policy=read('results/policy.json');calibration=read('results/calibration.jso
n');audit=read('data/audit.json')
 72     assert meta['classes']==m['classes']==selection['classes']==cfg['classes']==CLASSES
 73     assert not selection['test_accessed_during_selection'] and selection['scope_fitted']
 74     assert sha(ROOT/'configs/experiment.json')==meta['protocol_sha256']==selection['protocol_sha256']==audit['pr
otocol_sha256']
 75     assert
sha(ROOT/'data/manifest.csv')==meta['manifest_sha256']==selection['manifest_sha256']==audit['manifest_sha256']
 76     assert sha(ROOT/'results/selection.json')==calibration['selection_sha256']
 77     for name,expected in [('heads.npz',meta['heads_sha256']),('scope_head.npz',meta['scope_sha256']),('baseline.
npz',meta['baseline_sha256']),('mc3_features.onnx',read('models/encoder.json')['onnx_sha256']),('mobilenet_featu
res.onnx',read('models/focus_encoder.json')['onnx_sha256'])]:assert sha(ROOT/'models'/name)==expected,name
 78     assert sha(ROOT/'src/cricketshot/features.py')==meta['feature_source_sha256']
 79     assert sha(ROOT/'src/cricketshot/policy.py')==meta['decision_source_sha256']==policy['policy_code_sha256']
 80     assert sha(ROOT/'scripts/train.py')==selection['training_code_sha256']
 81     if 'class_followup' in selection:
 82         followup=selection['class_followup'];assert not followup['test_accessed']
 83         assert sha(ROOT/'configs/class_followup.json')==followup['protocol_sha256']
 84         assert sha(ROOT/'scripts/refine_classes.py')==followup['training_code_sha256']
 85     assert sha(ROOT/'scripts/calibrate.py')==calibration['calibration_code_sha256']
 86     assert sha(ROOT/'scripts/evaluate.py')==m['evaluation_code_sha256']
 87     rows=list(csv.DictReader((ROOT/'data/manifest.csv').open()));eligible=[r for r in rows if r['split'] in
['train','tune','calibration','policy','test']]
 88     assert len(rows)==audit['raw_clips']==sum(r['video_files'] for r in
read('data/provenance.json')['archives'])
 89     group_sets={s:{r['group_id'] for r in eligible if r['split']==s} for s in
['train','tune','calibration','policy','test']}
 90     for a in group_sets:
 91         for b in group_sets:
 92             if a!=b:assert not group_sets[a]&group_sets[b]
 93     assert all(r['label'] not in cfg['unseen_style_classes'] for r in eligible if r['split']!='test')
 94     assert len({(r['label'],r['clip_id']) for r in eligible})==len(eligible)
 95     with np.load(ROOT/'results/predictions.npz',allow_pickle=False) as z:e={k:z[k].copy() for k in z.files}
 96     y=e['y_true'];splits=e['split'];groups=e['group'];known=y>=0
 97     assert len(y)==len(eligible)
 98     for i,r in enumerate(eligible):
 99         assert e['path'][i]==r['path'] and splits[i]==r['split'] and groups[i]==int(r['group_id'])
100         assert y[i]==(CLASSES.index(r['target']) if r['target']!='review' else -1)
101     for name in selection['candidates']:
102         p=e[name+'__probabilities'];assert p.shape==(len(y),6) and
np.isfinite(p).all();np.testing.assert_allclose(p.sum(axis=1),1,atol=1e-12)
103         for split in group_sets:
104             mask=(splits==split)&known;same(arithmetic(y[mask],p[mask].argmax(axis=1)),m['models'][name][split],
name+'/'+split)
105         for split in
['train','tune']:same(m['models'][name][split],selection['candidates'][name][split],name+'/selection/'+split)
106     chosen=max(selection['candidates'],key=lambda n:selection['candidates'][n]['tune']['macro_f1']);assert
chosen==selection['deployment']==m['deployment']==meta['deployment']
107     assert [r['name'] for r in meta['members']]==selection['candidates'][chosen]['members']
108     same(meta['calibration'],calibration);same(policy['calibration'],calibration)
109     trace=read('results/training_trace.json')
110     for name,r in selection['heads'].items():
111         if r['head_type']=='mlp':
112             history=[h for h in trace if h['head']==name];best=max(h['tune_macro_f1'] for h in history);assert
r['best_epoch']==min(h['epoch'] for h in history if h['tune_macro_f1']==best)
113     assert selection['scope_C']==max(selection['scope_grid'],key=lambda r:r['tune_auroc'])['C']
114     if 'scope_followup' in selection:
115         followup=selection['scope_followup'];assert not followup['test_accessed']
116         assert sha(ROOT/'configs/scope_experiment.json')==followup['protocol_sha256']
117         assert sha(ROOT/'scripts/refine_scope.py')==followup['training_code_sha256']
118         best_scope=max(followup['candidates'],key=lambda
r:r['tune_auroc']);same(best_scope,followup['selected'])
119         assert meta['scope_representation']==best_scope['representation']
120         scope_trace=read('results/scope_trace.json')
```

scripts/verify.py

```
        for candidate in followup['candidates']:
            if candidate['head_type']=='mlp':
                history=[r for r in scope_trace if r['head']==candidate['name']];best_auc=max(r['tune_auroc'] for r in history)
                assert candidate['best_epoch']==min(r['epoch'] for r in history if r['tune_auroc']==best_auc)
                same(best_auc,candidate['tune_auroc'])
    p=e['conditional_raw'];temperature=calibration['temperature'];logits=np.log(np.maximum(p,1e-12))/temperature;logits-=logits.max(axis=1,keepdims=True);scaled=np.exp(logits);scaled/=scaled.sum(axis=1,keepdims=True)
    # The saved logit affine transform is float32; the stable sigmoid then
    # evaluates in float64. Preserve that boundary in this independent formula.
    affine=(calibration['scope_slope']*e['scope_logits']+calibration['scope_intercept']).astype(np.float64)
    scope=np.exp(-np.logaddexp(0,-affine));support=scope[:,None]*scaled
    np.testing.assert_allclose(scaled,e['conditional_calibrated'],atol=1e-12);np.testing.assert_allclose(scope,e['scope_score'],atol=1e-12);np.testing.assert_allclose(support,e['support'],atol=1e-12)
    cmask=splits=='calibration';ck=cmask&known;same(int(cmask.sum()),calibration['n']);same(int(ck.sum()),calibration['known'])
    original_logp=np.log(np.maximum(p[ck],1e-12));temperature_gradient=float(np.mean((original_logp[np.arange(ck.sum()),y[ck]]-(scaled[ck]*original_logp).sum(axis=1))/temperature))
    assert .05<=temperature<=20
    if .051<temperature<19.99:assert abs(temperature_gradient)<1e-4
    else:assert temperature_gradient>=-1e-4 if temperature<=.051 else temperature_gradient<=1e-4
    residual=scope[cmask]-known[cmask].astype(int)
    assert abs(float(residual.mean()))<1e-5
    assert abs(float(np.mean(residual*e['scope_logits'][cmask]))+calibration['scope_slope']/(100*cmask.sum()))<1e-5
    threshold=policy['threshold'];assert threshold==m['threshold']==meta['threshold']
    for split in group_sets:same(triage(y[splits==split],support[splits==split],threshold),m['triage'][split],split+'/triage')
    mask=splits=='policy';grid=[];target=cfg['policy_targets']
    for recorded in policy['grid']:
        r=triage(y[mask],support[mask],recorded['threshold']);same(r,recorded,'policy/grid');r['threshold']=recorded['threshold'];r['eligible']=bool(r['automatic_known']>=50 and r['known_automatic_accuracy']>=target['known_automatic_accuracy'] and r['out_of_scope_review_rate']>=target['exposure_review_rate']);r['fallback_objective']=r['known_coverage']*(r['known_automatic_accuracy'] or 0)-(1-r['out_of_scope_review_rate']);same(r['eligible'],recorded['eligible']);same(r['fallback_objective'],recorded['fallback_objective']);grid.append(r)
    candidates=[r for r in grid if r['eligible']]
    best=max(candidates,key=lambda r:(r['known_coverage'],r['known_automatic_accuracy'],-r['threshold'])) if candidates else max(grid,key=lambda r:(r['fallback_objective'],r['known_automatic_accuracy'] or 0,-r['threshold']))
    if 'operating_point' in policy:
        operational=read('configs/operating_point.json');recorded=policy['operating_point']
        assert sha(ROOT/'configs/operating_point.json')==recorded['config_sha256']
        assert sha(ROOT/'scripts/select_operating_point.py')==recorded['code_sha256']
        assert best['threshold']==operational['cautious_reference_threshold']
        best=next(r for r in grid if r['threshold']==operational['threshold'])
        goals=operational['development_goals'];same(recorded['development_goals_met'],best['known_coverage']>=goals['known_coverage'] and best['known_automatic_accuracy']>=goals['known_automatic_accuracy'] and best['out_of_scope_review_rate']>=goals['exposure_review_rate'])
    assert best['threshold']==threshold
    same(best,policy['policy_metrics']);same(best['eligible'],policy['accuracy_and_rejection_targets_met']);same(best['known_coverage']>=target['known_coverage_goal'],policy['known_coverage_goal_met'])
    base_grid=[dict(triage(y[mask],scaled[mask],t),threshold=t) for t in cfg['policy_grid']];matched=min(base_grid,key=lambda r:(abs(r['known_coverage']-best['known_coverage']),-(r['known_automatic_accuracy'] or 0),r['threshold']));assert matched['threshold']==policy['confidence_only_matched_coverage']['threshold'];same(matched,policy['confidence_only_matched_coverage']['policy_metrics'])
    test=splits=='test';same(triage(y[test],scaled[test],threshold),m['confidence_only_same_threshold']);same(triage(y[test],scaled[test],matched['threshold']),m['confidence_only_matched_coverage']['test'])
    unseen=np.isin(e['native_label'],cfg['unseen_style_classes']);same(triage(y[test&unseen],support[test&unseen],threshold),m['unseen_style_triage']);same(triage(y[test&~unseen&~known],support[test&~unseen&~known],threshold),m['exposure_style_triage'])
    majority=int(np.bincount(y[(splits=='train')&known],minlength=6).argmax());same(arithmetic(y[test&known],np.full(int((test&known).sum()),majority)),m['training_majority_baseline'])
```

scripts/verify.py

```
    for label in cfg['exposure_classes']+cfg['unseen_style_classes']:
        label_mask=test&(e['native_label']==label);same(triage(y[label_mask],support[label_mask],threshold),m['unknown_by_native_label'][label])
    for split in ['policy','test']:
        km=(splits==split)&known;actual={'raw':quality(y[km],p[km]),'scaled':quality(y[km],scaled[km])}
        if split=='policy':
            # Development and final evidence use different GEMM batch sizes.
            # Float32 roundoff can change losses by a few billionths. All
            # threshold counts/decisions above must still match exactly.
            for kind,key in [('raw','raw'),('scaled','temperature_scaled')]:
                for quantity,value in actual[kind].items():

np.testing.assert_allclose(value,policy['policy_calibration_quality'][key][quantity],rtol=0,atol=2e-7)
        else:same(actual['raw'],m['calibration_quality']['raw_test']);same(actual['scaled'],m['calibration_quality']['scaled_test'])
    same(auc(known[test],scope[test]),m['calibration_quality']['scope_test_auroc'])
    for split in ['train','tune','calibration','policy']:same(auc(known[splits==split],scope[splits==split]),policy['scope_auroc_by_role'][split])
    # Independent whole-group bootstrap, keeping the paired policies aligned.
    yy=y[test];gg=groups[test];ss=support[test];cc=scaled[test];ids=np.unique(gg);members=[np.flatnonzero(gg==g) for g in ids];rng=np.random.default_rng(m['bootstrap_seed']);draws=[]
    for _ in range(m['bootstrap_draws']):
        idx=np.concatenate([members[j] for j in rng.integers(0,len(ids),len(ids))]);km=yy[idx]>=0;r=triage(yy[idx],ss[idx],threshold);b=triage(yy[idx],cc[idx],matched['threshold']);raw=arithmetic(yy[idx][km],cc[idx][km].argmax(axis=1));draws.append([raw['accuracy'],raw['macro_f1'],r['known_coverage'],r['known_automatic_accuracy'] if r['known_automatic_accuracy'] is not None else np.nan,r['automatic_accuracy_all'] if r['automatic_accuracy_all'] is not None else
np.nan,r['out_of_scope_review_rate'],r['out_of_scope_review_rate']-b['out_of_scope_review_rate']])
    keys=['raw_known_accuracy','raw_known_macro_f1','known_coverage','known_automatic_accuracy','automatic_accuracy_all','out_of_scope_review_rate','review_gain_over_confidence_matched_coverage'];a=np.asarray(draws)
    for i,k in
enumerate(keys):same(np.nanquantile(a[:,i],[.025,.975]).tolist(),m['source_group_95ci'][k],k+'/ci')
    print(f'Passed {CHECKS} independent value checks, model selection, threshold grid, source separation and bootstrap intervals.',flush=True)
    if not args.replay:return
    from cricketshot.recognizer import Recognizer
    cv2.setNumThreads(1);recognizer=Recognizer();examples=read('demo/examples.json');lookup={r['path']:i for i,r in enumerate(eligible)};replays=[];times=[]
    for example in examples:
        path=ROOT/example['path'];assert
sha(path)==example['sha256'];result,frames=recognizer.predict(path);i=lookup[example['source_path']]
        assert example['sha256']==eligible[i]['sha256'] and example['source_label']==eligible[i]['label']
        values=np.array([result['support_scores'][c] for c in
CLASSES]);error=float(np.max(np.abs(values-support[i])));assert error<2e-4,(example['id'],error)
        expected=triage(y[i:i+1],support[i:i+1],threshold);assert
(result['decision']=='automatic')==bool(expected['automatic_all'])
        if result['decision']=='automatic':assert result['class']==CLASSES[int(support[i].argmax())]
        else:assert result['class'] is None
        assert result['input_sha256']==sha(path);times.append(result['elapsed_seconds']);replays.append({'id':example['id'],'decision':result['decision'],'class':result['class'],'input_sha256':result['input_sha256'],'score_max_abs_error':error,'seconds':result['elapsed_seconds']});print('Fresh
replay',example['id'],result['decision'],result['class'],flush=True)
    private=ROOT/'.build/runtime';private.mkdir(parents=True,exist_ok=True);pattern=private/'wiring_pattern.avi';writer=cv2.VideoWriter(str(pattern),cv2.VideoWriter_fourcc(*'FFV1'),24,(320,192));assert writer.isOpened()
    x=np.tile(np.arange(320,dtype=np.uint16),(192,1))
    for t in range(24):writer.write(np.stack([((x+t*7)%256).astype(np.uint8),((x//2+t*11)%256).astype(np.uint8),((x//3+t*5)%256).astype(np.uint8)],axis=-1))
    writer.release();pattern_sha=sha(pattern);wiring,_=recognizer.predict(pattern);assert
wiring['input_sha256']==pattern_sha
    record={'n_clips':len(times),'median_seconds':float(np.median(times)),'p95_seconds':float(np.quantile(times,.95)),'timing_scope':'Warm encoders: interval decode, motion, CNN features, scope and class heads. Excludes model initialisation, JPEG previews and browser
rendering.','examples':replays,'wiring_input':{'description':'Generated moving gradient; wiring/provenance check only, no recognition ground truth','input_sha256':pattern_sha,'recorded_sha256':wiring['input_sha256']},'heads_
```

scripts/verify.py

```
      sha256':meta['heads_sha256'],'independent_value_checks':CHECKS}
 197      destination=ROOT/'results/runtime_verification.json' if args.record else private/'latest_verification.json';
      destination.write_text(json.dumps(record,indent=2));print(json.dumps({'replayed':len(times),'median_seconds':rec
      ord['median_seconds'],'wiring_hash_matches':True},indent=2))
 198
 199
 200  if __name__=='__main__':main()
```

setup.bat

```
   1  @echo off
   2  cd /d "%~dp0"
   3  python -m venv .venv
   4  if errorlevel 1 exit /b 1
   5  ".venv\Scripts\python.exe" -m pip install -r requirements-lock.txt
   6  if errorlevel 1 exit /b 1
   7  echo Setup finished. Run run_demo.bat for the six-family video demo.
   8  pause
```

src/cricketshot/__init__.py

```
   1  """Six-family cricket recognition with an explicit review decision."""
   2  __version__="4.0.0"
```

src/cricketshot/features.py

```
   1  """Frozen video/frame features and camera-compensated temporal measurements.
   2
   3  Cropping is a fixed broadcast-layout prior, not a batter detector. Ordered bins
   4  have no contact/phase annotations. The same code serves extraction and the demo.
   5  """
   6  from pathlib import Path
   7  import hashlib,inspect,json
   8  import cv2,numpy as np,onnxruntime as ort
   9  from PIL import Image,ImageOps
  10  ROOT=Path(__file__).resolve().parents[2]
  11  MEAN=np.array([.43216,.394666,.37645],dtype=np.float32)
  12  STD=np.array([.22803,.22145,.216989],dtype=np.float32)
  13  BOX=(.20,.15,.80,1.0)
  14
  15  def normalize(x):
  16      a = np.asarray(x,dtype=np.float32)
  17      return a / np.maximum(np.linalg.norm(a,axis=-1,keepdims=True),1e-12)
  18
  19  def decode(path):
  20      """Bounded sequential decoding; a single trimmed shot, at most five seconds."""
  21      cap = cv2.VideoCapture(str(path))
  22      if not cap.isOpened():
  23          raise ValueError('Cannot decode this video. Use an AVI, MP4 or MOV clip.')
  24      fps = float(cap.get(cv2.CAP_PROP_FPS))
  25      frames = []
  26      try:
  27          while True:
  28              ok,b = cap.read()
  29              if not ok:
  30                  break
  31              if len(frames)>=600 or b.shape[0]*b.shape[1]>3840*2160:
  32                  raise ValueError('Please use a trimmed shot of at most five seconds, up to 4K.')
  33              h,w = b.shape[:2]
  34              b = cv2.resize(b,(320,max(32,round(h*320/w))),interpolation=cv2.INTER_AREA)
  35              frames.append(cv2.cvtColor(b,cv2.COLOR_BGR2RGB))
```

src/cricketshot/features.py

```
    finally:
        cap.release()
    if not frames or not np.isfinite(fps) or fps<=0:
        raise ValueError('No decodable frames or valid frame rate.')
    if len(frames)/fps>5.05:
        raise ValueError('Please trim the video to one shot of at most five seconds.')
    if len({f.shape for f in frames})!=1:
        raise ValueError('Changing frame geometry is not supported.')
    return np.stack(frames), {'decoded_frames':len(frames),'fps':fps,'duration_seconds':len(frames)/fps}

def sample_indices(n,start=0,stop=None,count=24):
    stop = n if stop is None else stop
    if not 0<=start<stop<=n:
        raise ValueError('Invalid temporal window')
    return np.rint(np.linspace(start,stop-1,count)).astype(int)

def prepare(frames,indices):
    """Official MC3 resize(128,171), centre crop112, Kinetics normalisation.

    cv2 uses bilinear interpolation; the exported encoder sees precisely this
    preprocessing in both feature extraction and interactive inference.
    """
    a = np.stack([cv2.resize(frames[i],(171,128),interpolation=cv2.INTER_LINEAR)[8:120,30:142] for i in
indices])
    a = (a.astype(np.float32)/255-MEAN)/STD
    return np.ascontiguousarray(a.transpose(3,0,1,2)[None])

def activity(frames):
    """Estimate residual motion after robust background affine alignment.

    Exclude the central batter area from camera fitting. Failed fits, hard cuts
    and tiny activity are reported and trigger a uniform-sampling fallback.
    """
    gray =
[cv2.resize(cv2.cvtColor(f,cv2.COLOR_RGB2GRAY),(160,max(16,round(f.shape[0]/2))),interpolation=cv2.INTER_AREA)
for f in frames]
    h,w = gray[0].shape
    region = (slice(round(.05*h),round(.85*h)),slice(round(.30*w),round(.70*w)))
    mask = np.full((h,w),255,np.uint8)
    mask[round(.02*h):round(.94*h),round(.24*w):round(.76*w)] = 0
    raw,comp,reliable,cuts = [],[],[],[]
    for prev,cur in zip(gray[:-1],gray[1:]):
        points = cv2.goodFeaturesToTrack(prev,160,.01,5,mask=mask)
        matrix,okfit = None,False
        if points is not None and len(points)>=8:
            nxt,status,_ = cv2.calcOpticalFlowPyrLK(prev,cur,points,None,winSize=(15,15),maxLevel=2)
            if nxt is not None:
                good = status.reshape(-1).astype(bool)
                a,b = points[good].reshape(-1,2),nxt[good].reshape(-1,2)
                if len(a)>=8:
                    # Seed makes OpenCV RANSAC repeatable across extractor/demo.
                    cv2.setRNGSeed(604)
                    matrix,inliers =
cv2.estimateAffinePartial2D(a,b,method=cv2.RANSAC,ransacReprojThreshold=2,maxIters=1000)
                    if matrix is not None and inliers is not None:
                        scale = float(np.hypot(matrix[0,0],matrix[0,1]))
                        okfit = float(inliers.mean())>=.5 and .9<=scale<=1.1
        flow = cv2.calcOpticalFlowFarneback(prev,cur,None,.5,2,11,2,5,1.1,0)
        mag = np.linalg.norm(flow[region],axis=-1)
        raw.append(float(np.quantile(mag,.8)))
        if okfit:
            aligned = cv2.warpAffine(prev,matrix,(w,h),flags=cv2.INTER_LINEAR,borderMode=cv2.BORDER_REFLECT_101)
            cut = float(np.mean(np.abs(aligned.astype(np.float32)-cur)))>45
            residual = cv2.calcOpticalFlowFarneback(aligned,cur,None,.5,2,11,2,5,1.1,0)
```

src/cricketshot/features.py

```
            comp.append(float(np.quantile(np.linalg.norm(residual[region],axis=-1),.8)) if not cut else 0.)
        else:
            cut = float(np.mean(np.abs(prev.astype(np.float32)-cur)))>45
            comp.append(0.)
        reliable.append(bool(okfit and not cut))
        cuts.append(bool(cut))
    return {'raw':raw,'compensated':comp,'reliable':reliable,'cuts':cuts}

def windows(frames):
    n = len(frames)
    motion = activity(frames)
    length = max(2,min(n,int(np.ceil(.75*n))))
    center = (n-length)//2
    uniform = np.arange(n)
    selected = {'uniform':uniform,'center':np.arange(center,center+length)}
    details = {}
    for key,scores in [('raw_motion',motion['raw']),('compensated_motion',motion['compensated'])]:
        a = np.asarray(scores,float)
        candidates = np.array([a[i:i+length-1].mean() for i in range(n-length+1)]) if len(a) else np.array([0.])
        start = int(candidates.argmax())
        reliable = key=='raw_motion' or (np.mean(motion['reliable'])>=.5 if len(a) else False)
        meaningful = bool(candidates.max()>=.05 and candidates.max()>1.05*(candidates.mean()+1e-8))
        fallback = not (reliable and meaningful)
        selected[key] = uniform.copy() if fallback else np.arange(start,start+length)
        details[key] = {'start':0 if fallback else start,'stop':n if fallback else start+length,
                        'fallback_uniform':fallback,'window_score':float(candidates.max()),
                        'reliable_fraction':float(np.mean(motion['reliable'])) if len(a) else 0.}
    return selected, {'activity':motion,'windows':details}

class VideoEncoder:
    def __init__(self,path=None,threads=4):
        model=Path(path or ROOT/'models/mc3_features.onnx')
        if not model.is_file():raise FileNotFoundError('Missing video encoder.')
        options = ort.SessionOptions()
        options.intra_op_num_threads=threads;options.inter_op_num_threads=1
        self.session = ort.InferenceSession(str(model),sess_options=options,providers=['CPUExecutionProvider'])
        self.input = self.session.get_inputs()[0].name

    def encode(self,frames):
        """Return (24,512) features with spatial pooling, before temporal pooling."""
        return
self.session.run(None,{self.input:prepare(frames,np.arange(len(frames)))})[0][0].astype(np.float32)

    def views(self,frames):
        frames = frames[sample_indices(len(frames))]
        selected,details = windows(frames)
        sequence = self.encode(frames)
        features = {key:sequence[indices].mean(axis=0) for key,indices in selected.items()}
        features['sequence'] = sequence
        return features,details,selected

def decode_focused(path):
    """Crop original decoded pixels before reducing resolution for the frame CNN."""
    cap=cv2.VideoCapture(str(path));frames=[];fps=float(cap.get(cv2.CAP_PROP_FPS))
    try:
        while True:
            ok,bgr=cap.read()
            if not ok:break
            if len(frames)>=600 or bgr.shape[0]*bgr.shape[1]>3840*2160:raise ValueError('Use a short clip up to
4K.')
            h,w=bgr.shape[:2];x1,y1,x2,y2=BOX
            rgb=cv2.cvtColor(bgr[round(y1*h):round(y2*h),round(x1*w):round(x2*w)],cv2.COLOR_BGR2RGB)
            h,w=rgb.shape[:2];scale=min(1.,320/max(h,w))
            rgb=cv2.resize(rgb,(max(1,round(w*scale)),max(1,round(h*scale))),interpolation=cv2.INTER_AREA)
```

src/cricketshot/features.py

```
            frames.append(rgb)
    finally:cap.release()
    if not frames or not np.isfinite(fps) or fps<=0 or len(frames)/fps>5.05:raise ValueError('Use one shot, at most five seconds.')
    return np.stack(frames)

def prepare_focused(frames):
    images=[np.asarray(ImageOps.fit(Image.fromarray(f),(224,224),method=Image.Resampling.BILINEAR),dtype=np.float32)/255 for f in frames]
    x=(np.stack(images)-np.array([.485,.456,.406],np.float32))/np.array([.229,.224,.225],np.float32)
    return np.ascontiguousarray(x.transpose(0,3,1,2))

class FrameEncoder:
    """Frozen ImageNet MobileNetV2, used on every selected focused frame."""
    def __init__(self,path=None):
        root=Path(__file__).resolve().parents[2]
        model=Path(path or root/'models/mobilenet_features.onnx')
        if not model.is_file():raise FileNotFoundError('Missing focused frame encoder.')
        opts=ort.SessionOptions();opts.intra_op_num_threads=2;opts.inter_op_num_threads=1
        self.session=ort.InferenceSession(str(model),sess_options=opts,providers=['CPUExecutionProvider'])
        self.input=self.session.get_inputs()[0].name
    def encode(self,frames):
        x=prepare_focused(frames)
        return np.concatenate([self.session.run(None,{self.input:x[i:i+8]})[0].reshape(-1,1280) for i in range(0,len(x),8)]).astype(np.float32)

def describe(video,focused,indices,representation):
    """Pool fixed features; all fitted projection/scaling is in the learned head."""
    g=normalize(video.mean(axis=0))
    if representation=='video_mean':return g
    context=[g,normalize(focused.mean(axis=0))]
    if representation=='context_mean':return normalize(np.concatenate(context))
    if representation not in {'full_ordered','motion_ordered'}:raise ValueError('Unknown representation')
    chosen=indices['uniform' if representation=='full_ordered' else 'compensated_motion']
    bins=[normalize(focused[b].mean(axis=0)) for b in np.array_split(chosen,3)]
    return normalize(np.concatenate([*context,*bins]))


def feature_binding(root=ROOT):
    root=Path(root)
    parts=[(root/'data/manifest.csv').read_bytes(),Path(__file__).read_bytes()]
    for name in ['mc3_features.onnx','mobilenet_features.onnx']:
        with (root/'models'/name).open('rb') as f:parts.append(hashlib.file_digest(f,'sha256').digest())
    return hashlib.sha256(b''.join(parts)).hexdigest()
```

src/cricketshot/policy.py

```
"""Numeric classifiers, calibrated class support and explicit candidate-set review."""
import numpy as np


CLASSES=['drive','defence','flick','pull','square_cut','sweep']
DISPLAY={c:c.replace('_',' ').title() for c in CLASSES}


def head_logits(feature,head):
    x=(np.asarray(feature,dtype=np.float32)-head['mean'])/head['scale']
    if 'input_clip' in head:x=np.clip(x,-float(head['input_clip']),float(head['input_clip']))
    if 'projection' in head:x=(x-head['projection_center'])@head['projection'].T
    if 'weight1' in head:x=np.maximum(0,x@head['weight1'].T+head['bias1'])
    return x@head['weight'].T+head['bias']


```

src/cricketshot/policy.py

```
def softmax(logits):
    z=np.asarray(logits,dtype=np.float64)
    z=z-z.max(axis=-1,keepdims=True);e=np.exp(z)
    return e/e.sum(axis=-1,keepdims=True)


def sigmoid(logits):
    z=np.asarray(logits,dtype=np.float64)
    return np.exp(-np.logaddexp(0,-z))


def calibrated_support(conditional,scope_logits,calibration):
    """Joint support = calibrated scope score * conditional six-family score.

    This is a learned score under the calibration mixture, not a guarantee that
    an unusual video is recognised as unknown or that a human will be correct.
    """
    p=np.asarray(conditional,dtype=np.float64)
    t=float(calibration['temperature'])
    if t<=0 or not np.isfinite(p).all() or np.any(p<0):raise ValueError('Invalid class scores')
    cp=softmax(np.log(np.maximum(p,1e-12))/t)
    scope=sigmoid(float(calibration['scope_slope'])*np.asarray(scope_logits)+float(calibration['scope_intercept']))
    return cp*np.expand_dims(scope,-1),cp,scope


def decide(support,threshold,classes=CLASSES):
    scores=np.asarray(support,dtype=np.float64)
    if scores.shape!=(len(classes),) or not np.isfinite(scores).all() or np.any(scores<0) or np.any(scores>1):raise ValueError('Invalid support scores')
    if not 0<threshold<1:raise ValueError('Invalid support threshold')
    selected=np.flatnonzero(scores>threshold).tolist()
    reason='exactly_one_candidate' if len(selected)==1 else 'no_candidate' if not selected else 'multiple_candidates'
    return {'decision':'automatic' if len(selected)==1 else 'human_review',
            'class':classes[selected[0]] if len(selected)==1 else None,
            'candidate_classes':[classes[i] for i in selected],
            'reason':reason,'top_candidate':classes[int(scores.argmax())],
            'support_scores':{c:float(s) for c,s in zip(classes,scores)},'threshold':float(threshold)}
```

src/cricketshot/recognizer.py

```
"""Six-family recognition with a calibrated support threshold and review outcome."""
from pathlib import Path
import hashlib,json,time
import cv2,numpy as np
from .features import ROOT,BOX,VideoEncoder,FrameEncoder,sample_indices,windows,describe
from .policy import CLASSES,DISPLAY,head_logits,softmax,calibrated_support,decide


def file_sha(path):
    with Path(path).open('rb') as f:return hashlib.file_digest(f,'sha256').hexdigest()


def decode_interval(path,start=0.,end=None):
    """Decode once, reproducing full/focused training pixels for a selected interval.

    A longer upload is allowed, but only one interval up to five seconds is
    classified. Sequential decoding preserves the frame-index provenance.
    """
    start=float(start);end=None if end is None else float(end)
    if not np.isfinite(start) or start<0 or start>120:raise ValueError('Start must be between 0 and 120 seconds.')
```

src/cricketshot/recognizer.py

```
    if end is not None and (not np.isfinite(end) or end<=start or end-start>5.05 or end>120):raise ValueError('Select one batting interval of at most five seconds.')
    cap=cv2.VideoCapture(str(path));full=[];focused=[]
    if not cap.isOpened():cap.release();raise ValueError('Cannot decode this video. Try another video encoding.')
    fps=float(cap.get(cv2.CAP_PROP_FPS));source_count=int(max(0,cap.get(cv2.CAP_PROP_FRAME_COUNT)))
    if not np.isfinite(fps) or fps<=0 or fps>240:cap.release();raise ValueError('The video has an invalid or unsupported frame rate.')
    source_duration=source_count/fps if source_count else None
    if source_duration is not None and source_duration>120.05:cap.release();raise ValueError('Upload up to two minutes, then select a batting interval.')
    if end is None and source_duration is not None and source_duration-start>5.05:cap.release();raise ValueError('Choose a start and end around one shot, at most five seconds.')
    first=int(np.floor(start*fps+1e-8));last=None if end is None else int(np.ceil(end*fps-1e-8));shape=None;index=0
    try:
        while index<28800:
            if last is not None and index>=last:break
            ok,bgr=cap.read()
            if not ok:break
            if index<first:index+=1;continue
            if len(full)>=600 or bgr.shape[0]*bgr.shape[1]>3840*2160:raise ValueError('Use at most 600 selected frames and resolution up to 4K.')
            if shape is None:shape=bgr.shape
            if bgr.shape!=shape:raise ValueError('Changing frame geometry is not supported.')
            h,w=bgr.shape[:2];small=cv2.resize(bgr,(320,max(32,round(h*320/w))),interpolation=cv2.INTER_AREA)
            full.append(cv2.cvtColor(small,cv2.COLOR_BGR2RGB))

x1,y1,x2,y2=BOX;rgb=cv2.cvtColor(bgr[round(y1*h):round(y2*h),round(x1*w):round(x2*w)],cv2.COLOR_BGR2RGB)
            h,w=rgb.shape[:2];scale=min(1.,320/max(h,w));focused.append(cv2.resize(rgb,(max(1,round(w*scale)),max(1,round(h*scale))),interpolation=cv2.INTER_AREA));index+=1
    finally:cap.release()
    if not full:raise ValueError('No decodable frames in the selected interval.')
    if len(full)/fps>5.05:raise ValueError('Select at most five seconds around one batting shot.')
    return np.stack(full),np.stack(focused),{'fps':fps,'selected_frames':len(full),'duration_seconds':len(full)/fps,'start_seconds':first/fps,'end_seconds':(first+len(full))/fps,'source_reported_frames':source_count,'source_reported_duration_seconds':source_duration},first


class Recognizer:
    def __init__(self,root=ROOT):
        cv2.setNumThreads(1)

self.root=Path(root);models=self.root/'models';self.meta=json.loads((models/'classifier.json').read_text())
        if self.meta['schema_version']!=4 or self.meta['classes']!=CLASSES:raise ValueError('Classifier taxonomy is inconsistent.')
        for path,key in [('heads.npz','heads_sha256'),('scope_head.npz','scope_sha256')]:
            if file_sha(models/path)!=self.meta[key]:raise ValueError('Classifier weight hash mismatch.')
        with np.load(models/'heads.npz',allow_pickle=False) as z:self.heads={m['name']:{k.split('__',1)[1]:z[k].copy() for k in z.files if k.startswith(m['name']+'__')} for m in self.meta['members']}
        with np.load(models/'scope_head.npz',allow_pickle=False) as z:self.scope={k:z[k].copy() for k in z.files}
        for h in [*self.heads.values(),self.scope]:
            if np.any(h['scale']<=0) or not all(np.isfinite(a).all() for a in h.values()):raise ValueError('Invalid numerical head parameters.')
        self.encoder=VideoEncoder(models/'mc3_features.onnx');self.frame_encoder=FrameEncoder(models/'mobilenet_features.onnx')

    def predict(self,path,start=0.,end=None):
        began=time.perf_counter();full,focused,info,first=decode_interval(path,start,end);ids=sample_indices(len(full));full=full[ids];focused=focused[ids]
        indices,motion=windows(full);video_features=self.encoder.encode(full);frame_features=self.frame_encoder.encode(focused);representations={}
```

src/cricketshot/recognizer.py

```
        values=[];member_predictions=[]
        for m in self.meta['members']:
            rep=m['representation']
            if rep not in
representations:representations[rep]=describe(video_features,frame_features,indices,rep)
            probabilities=softmax(head_logits(representations[rep],self.heads[m['name']]))
            values.append(probabilities);member_predictions.append({'name':m['name'],'representation':rep,'class
':CLASSES[int(probabilities.argmax())]})
        conditional=np.mean(values,axis=0);scope_rep=self.meta.get('scope_representation','context_mean')
        if scope_rep not in
representations:representations[scope_rep]=describe(video_features,frame_features,indices,scope_rep)
        scope_logit=float(head_logits(representations[scope_rep],self.scope).ravel()[0])
        support,scaled,scope_score=calibrated_support(conditional,scope_logit,self.meta['calibration']);result=d
ecide(support,self.meta['threshold'])
        result.update(display_name=DISPLAY[result['class']] if result['class'] else 'Human
review',conditional_scores={c:float(s) for c,s in zip(CLASSES,scaled)},scope_score=float(scope_score),model=self
.meta['name'],deployment=self.meta['deployment'],input_sha256=file_sha(path),heads_sha256=self.meta['heads_sha25
6'],video=info,encoder_sample_indices=(first+ids).tolist(),member_predictions=member_predictions,motion=motion,f
ocus_box_xyxy_normalized=list(BOX),score_note='Calibrated support under the development data mixture. Scores do
not guarantee correctness or unknown-shot detection.',elapsed_seconds=time.perf_counter()-began)
        return result,{'full':full,'focused':focused}
```

static/app.css

```
:root{--paper:#f7f8f2;--ink:#20372b;--green:#2e5638;--muted:#69766b;--line:#d9dfd3;--amber:#986022}
*{box-sizing:border-box} [hidden]{display:none!important}
body{margin:0;background:var(--paper);color:var(--ink);font:15px/1.6 "Segoe UI",Arial,sans-serif}
a{color:var(--green)} button,input,select,textarea{font:inherit}
button{cursor:pointer}button:disabled{cursor:default;opacity:.45}
header{max-width:1240px;margin:auto;padding:25px 28px;border-bottom:1px solid
var(--line);display:flex;justify-content:space-between;align-items:center}
.wordmark{font:28px Georgia,serif;letter-spacing:-1px;text-decoration:none;color:var(--ink)}.wordmark
span{font-weight:bold}.local{font-size:12px;color:var(--muted)}
main{max-width:1240px;margin:auto;padding:0 28px}.hero{max-width:780px;padding:52px 0
42px}.eyebrow{font-size:11px;letter-spacing:2px;font-weight:600}.hero h1{font:58px/1.05
Georgia,serif;letter-spacing:-2px;margin:20px 0 22px}.hero h1
em{color:#5a7850}.intro{font-size:18px;max-width:680px}.labels{margin:24px 0
5px;font-size:15px}.muted{color:var(--muted);font-size:12px}.workspace{display:grid;grid-template-columns:1fr
1.12fr;background:#fff;border:1px solid
var(--line)}.input-section,.result-section{padding:28px}.input-section{border-right:1px solid
var(--line)}h2{font-size:21px;letter-spacing:-.4px;margin:0 0
25px}.step{display:inline-flex;align-items:center;justify-content:center;border:1px solid var(--line);border-rad
ius:50%;width:31px;height:31px;margin-right:12px;font-size:10px;color:var(--muted);font-weight:400}h3{font-size:
17px}h4{font-size:14px;margin:20px 0 8px}
.dropzone{display:flex;flex-direction:column;align-items:center;justify-content:center;border:1px dashed
#cbd4c3;background:#fafbf7;padding:30px 15px;min-height:190px;cursor:pointer;text-align:center}.dropzone.dragove
r{border-color:var(--green);background:#eef3e7}.dropzone strong{overflow-wrap:anywhere}.dropzone
span:not(.plus),.dropzone small{color:var(--muted);font-size:12px}.dropzone
.plus{font-size:30px;color:var(--green);font-weight:300}.dropzone
small{margin-top:18px;font-size:10px}.primary-button{width:100%;padding:12px 16px;border:0;background:var(--gree
n);color:#fff;margin-top:14px;text-align:left;display:flex;justify-content:space-between}.input-note{margin-top:
15px}.time-controls{display:grid;grid-template-columns:1fr 1fr;gap:12px}.time-controls label,.human-review
label{display:block;font-size:12px;margin:10px 0}.time-controls input,.human-review select,.human-review
textarea{width:100%;padding:8px;border:1px solid
var(--line);border-radius:0;background:#fff;color:var(--ink)}.time-buttons{display:flex;gap:12px}.time-buttons
button,.text-button{background:none;border:0;color:var(--green);padding:4px
0;text-decoration:underline;text-underline-offset:4px;font-size:12px}.time-buttons button{font-size:11px}#video-
preview{display:block;width:100%;max-height:240px;background:#101b13;margin-top:15px}.examples-title{margin:30px
 0 0}.examples-title+p{margin-top:2px}.examples{display:grid;grid-template-columns:repeat(3,1fr);gap:9px}.exampl
e{background:#fff;border:1px solid
var(--line);padding:0;text-align:left;color:var(--ink);font-size:11px}.example
img{display:block;width:100%;height:67px;object-fit:cover}.example
span{display:block;padding:4px}.example.active{outline:2px solid
```

static/app.css

```
#7c945d;outline-offset:1px}.empty-state,.loading-state{text-align:center;padding:70px
12px;color:var(--muted)}.empty-state>strong{font:70px Georgia,serif;color:#80946f}.empty-state
h3{color:var(--ink)}.spinner{display:inline-block;width:26px;height:26px;border:2px solid
var(--line);border-top-color:var(--green);border-radius:50%;animation:spin .9s linear infinite}@keyframes
spin{to{transform:rotate(360deg)}}.error-state{padding:18px;background:#fff0e5;color:#7b3e1c;border:1px solid
#ebc8af}.shot-label{font:38px/1.15 Georgia,serif;margin:8px 0 15px;letter-spacing:-1px}.review
.shot-label,.review #decision-label{color:var(--amber)}#decision-note{font-size:14px}#example-note{margin:12px
0}.threshold-note{padding:10px 12px;background:#f0f4e9;font-size:12px;margin:22px 0 12px}.review
.threshold-note{background:#faf2e7}.scores{display:grid;grid-template-columns:1fr 1fr;gap:13px 18px}.class-row{f
ont-size:12px}.class-line{display:flex;justify-content:space-between;gap:6px}.class-row.passes{font-weight:600;c
olor:var(--green)}.class-track{height:4px;background:#edf0e7;position:relative;margin-top:5px}.class-fill{height
:100%;background:#aac09a}.passes .class-fill{background:var(--green)}.threshold-mark{position:absolute;top:-3px;
height:10px;width:1px;background:var(--amber)}.human-review{margin:26px 0;padding:10px 0 18px;border-top:2px
solid #cda365;border-bottom:1px solid var(--line)}.human-review
h4{color:var(--amber);margin-top:10px}.human-review p{font-size:12px}.secondary-button{border:1px solid
var(--green);padding:8px 12px;background:#fff;color:var(--green);font-size:12px}.frames-title{display:flex;justi
fy-content:space-between;align-items:baseline;gap:12px;margin-top:28px}.frame-strip{display:grid;grid-template-c
olumns:repeat(4,1fr);gap:5px;margin:8px 0 13px}.frame-strip
img{width:100%;height:65px;object-fit:cover}.frames-title h4{margin-top:0}.frames-title
span{white-space:nowrap}details{border-top:1px solid var(--line);padding:13px
0;margin-top:15px;font-size:12px}summary{cursor:pointer;color:var(--ink);font-size:13px}details p{color:var(--mu
ted)}canvas{width:100%;height:auto;margin-top:15px}.conditional-scores{font-size:12px;color:var(--muted)}.text-b
utton{margin-top:20px}.evidence{padding:48px 0;border-bottom:1px solid var(--line)}.evidence
h2{font-size:28px;margin:10px
0}.evidence>p{max-width:780px}.metrics{display:grid;grid-template-columns:repeat(4,1fr);gap:18px;margin:30px
0}.metric strong{display:block;color:var(--green);font:33px Georgia,serif}.metric
span{display:block;font-size:12px;color:var(--muted);margin-top:6px}.evidence details{max-width:900px}.evidence
details p{font-size:13px}footer{display:flex;justify-content:space-between;gap:35px;padding:24px 0
30px;color:var(--muted);font-size:11px}footer span:first-child{max-width:850px}
@media(max-width:850px){.workspace{grid-template-columns:1fr}.input-section{border-right:0;border-bottom:1px
solid var(--line)}.hero{padding-top:32px}.hero
h1{font-size:49px}.metrics{grid-template-columns:repeat(2,1fr)}.examples img{height:80px}}
@media(max-width:480px){header,main{padding-left:20px;padding-right:20px}.local{font-size:10px}.input-section,.r
esult-section{padding:22px}.hero
h1{font-size:45px}.intro{font-size:16px}.examples{grid-template-columns:repeat(2,1fr)}.scores{gap:13px
12px}.frame-strip img{height:54px}footer{flex-direction:column;gap:12px}}
```

static/app.js

```
/* Upload/interval selection and the fixed model policy. No threshold overrides. */
const $=id=>document.getElementById(id);
const names={drive:'Drive',defence:'Defence',flick:'Flick',pull:'Pull',square_cut:'Square cut',sweep:'Sweep'};
let selectedFile=null,videoURL=null,lastResult=null;
const pct=x=>Number.isFinite(x)?`${(100*x).toFixed(1)}%`:'—';
function error(message){$('loading-state').hidden=true;$('prediction').hidden=true;$('empty-state').hidden=true;
$('error-state').hidden=false;$('error-state').textContent=message;}
function busy(active){$('analyze-button').disabled=active||!selectedFile;for(const b of
document.querySelectorAll('.example'))b.disabled=active;}
async function request(url,body){busy(true);$('empty-state').hidden=true;$('prediction').hidden=true;$('error-st
ate').hidden=true;$('loading-state').hidden=false;try{const r=await fetch(url,{method:'POST',body});let
data;try{data=await r.json();}catch{throw new Error('The server could not return a result. Check that the local
app is running.');}if(!r.ok)throw new Error(data.error||'The video could not be
analysed.');render(data);}catch(e){error(e.message);}finally{busy(false);}}
function choose(file){if(!file)return;if(file.size>=100*1024*1024){error('Use a video below 100
MB.');return;}if(!/\.(mp4|avi|mov|mkv|webm)$/i.test(file.name)){error('Choose MP4, AVI, MOV, MKV or WebM.');retu
rn;}selectedFile=file;$('file-label').textContent=file.name;$('trim-panel').hidden=false;$('start').value='0';$(
'end').value='';$('analyze-button').disabled=false;if(videoURL)URL.revokeObjectURL(videoURL);videoURL=URL.create
ObjectURL(file);$('video-preview').src=videoURL;$('preview-note').textContent='Select the preparation, stroke
and follow-through, up to five seconds.';}
$('video-input').addEventListener('change',e=>choose(e.target.files[0]));
for(const event of ['dragenter','dragover'])$('dropzone').addEventListener(event,e=>{e.preventDefault();$('dropz
one').classList.add('dragover');});
for(const event of ['dragleave','drop'])$('dropzone').addEventListener(event,e=>{e.preventDefault();$('dropzone'
```

static/app.js

```
).classList.remove('dragover');if(event==='drop')choose(e.dataTransfer.files[0]);});
$('video-preview').addEventListener('loadedmetadata',()=>{const duration=$('video-preview').duration;if(Number.isFinite(duration)){$('end').value=Math.min(duration,5).toFixed(2);$('preview-note').textContent=duration>5?'Choose up to five seconds around the batting action; the remaining footage will not be classified.':'Check that the interval contains one complete batting action.';}});
$('video-preview').addEventListener('error',()=>{$('preview-note').textContent='Browser preview is unavailable for this encoding. Enter an interval, or leave End blank for a clip already shorter than five seconds.';});
$('mark-start').addEventListener('click',()=>{$('start').value=($('video-preview').currentTime||0).toFixed(2);});
$('mark-end').addEventListener('click',()=>{$('end').value=($('video-preview').currentTime||0).toFixed(2);});
$('upload-form').addEventListener('submit',e=>{e.preventDefault();if(!selectedFile)return;const start=Number($('start').value||0),end=$('end').value?Number($('end').value):null;if(!Number.isFinite(start)||start<0||(end!==null&&(!Number.isFinite(end)||end<=start||end-start>5.05))){error('Choose a valid batting interval of at most five seconds.');return;}const body=new FormData();body.append('video',selectedFile);body.append('start',String(start));if(end!==null)body.append('end',String(end));request('/api/predict',body);});
function frames(id,images){$(id).replaceChildren();images.forEach((src,i)=>{const im=document.createElement('img');im.src=src;im.alt=`Sampled model frame ${i+1}`;$(id).append(im);});}
function activity(motion){const c=$('activity-chart'),g=c.getContext('2d'),w=c.width,h=c.height;g.clearRect(0,0,w,h);g.fillStyle='#fafbf7';g.fillRect(0,0,w,h);const series=[motion.activity.raw,motion.activity.compensated];const max=Math.max(.01,...series.flat());for(let k=0;k<2;k++){const values=series[k];g.strokeStyle=k?'#b37532':'#2e5638';g.lineWidth=2;g.beginPath();values.forEach((v,i)=>{const x=28+i*(w-56)/Math.max(1,values.length-1),y=h-35-v*(h-65)/max;i?g.lineTo(x,y):g.moveTo(x,y);});g.stroke();}g.font='13px Segoe UI';g.fillStyle='#2e5638';g.fillText('Raw motion',28,18);g.fillStyle='#b37532';g.fillText('After compensation',150,18);g.fillStyle='#69766b';g.fillText('Start',28,h-10);g.fillText('End',w-55,h-10);}
function render(data){lastResult=data;$('loading-state').hidden=true;$('prediction').hidden=false;$('prediction').classList.toggle('review',data.decision==='human_review');$('decision-label').textContent=data.decision==='automatic'?'AUTOMATIC LABEL':'HUMAN REVIEW';$('shot-label').textContent=data.display_name;$('decision-note').textContent=data.reason==='exactly_one_candidate'?'Exactly one shot family passed the fixed threshold.':data.reason==='multiple_candidates'?`More than one family passed: ${data.candidate_classes.map(c=>names[c]).join(', ')}. Inspect the movement before assigning a label.`:'No shot family passed the fixed threshold. Inspect the movement before assigning a label.';$('example-note').textContent=data.example?`Source label: ${data.example.source_label.replaceAll('_',' ')}. ${data.example.note}`:'Prediction for your selected video interval';$('threshold-note').textContent=`Threshold ${pct(data.threshold)} / ${data.candidate_classes.length} passing ${data.candidate_classes.length===1?'class':'classes'} / amber marks show the threshold.`;
  $('scores').replaceChildren();for(const [label,score] of Object.entries(data.support_scores).sort((a,b)=>b[1]-a[1])){const row=document.createElement('div');row.className=`class-row${score>data.threshold?' passes':''}`;const line=document.createElement('div');line.className='class-line';const name=document.createElement('span'),number=document.createElement('span');name.textContent=names[label];number.textContent=pct(score);line.append(name,number);const track=document.createElement('div');track.className='class-track';const fill=document.createElement('div');fill.className='class-fill';fill.style.width=pct(score);const mark=document.createElement('span');mark.className='threshold-mark';mark.style.left=pct(data.threshold);track.append(fill,mark);row.append(line,track);$('scores').append(row);}
  $('human-review').hidden=data.decision!=='human_review';$('review-label').value='';$('review-notes').value='';$('download-review').disabled=true;frames('focused-frames',data.previews.focused);frames('full-frames',data.previews.full);$('timing').textContent=`${data.elapsed_seconds.toFixed(2)} s on CPU`;$('frame-note').textContent=`${data.video.selected_frames} frames in ${data.video.duration_seconds.toFixed(2)} s, source interval ${data.video.start_seconds.toFixed(2)}–${data.video.end_seconds.toFixed(2)} s. The previews show actual sampled inputs.`;activity(data.motion);const motion=data.motion.windows.compensated_motion;$('motion-note').textContent=`Camera fits reliable on ${pct(motion.reliable_fraction)} of sampled transitions. The motion selector ${motion.fallback_uniform?'used its full-clip fallback':'selected a motion-heavy interval'}. Activity units are pixels per sampled transition at 160-pixel analysis width, without contact annotations.`;$('scope-note').textContent=`Scope score: ${pct(data.scope_score)}. It measures support for belonging to the six trained families.`;$('conditional-scores').textContent=Object.entries(data.conditional_scores).sort((a,b)=>b[1]-a[1]).map(([c,p])=>`${names[c]} ${pct(p)}`).join(' / ');
}
function download(value,name){const blob=new Blob([JSON.stringify(value,null,2)],{type:'application/json'}),url=URL.createObjectURL(blob),a=document.createElement('a');a.href=url;a.download=name;a.click();setTimeout(()=>URL.revokeObjectURL(url),1000);}
$('download-result').addEventListener('click',()=>{if(lastResult){const
```

static/app.js

```
{previews,...record}=lastResult;download(record,'cricketshot-prediction.json');}});
$('review-label').addEventListener('change',()=>{$('download-review').disabled=!$('review-label').value;});
$('download-review').addEventListener('click',()=>{if(!lastResult||!$('review-label').value)return;const {previews,...record}=lastResult;download({...record,human_review:{label:$('review-label').value,observation:$('review-notes').value,recorded_utc:new Date().toISOString()}},'cricketshot-reviewed.json');});
async function load(){try{const [examples,evidence]=await Promise.all([fetch('/api/examples').then(r=>r.json()),fetch('/api/evidence').then(r=>r.json())]);for(const e of examples){const b=document.createElement('button');b.type='button';b.className='example';const im=document.createElement('img');im.src=`/api/example/${e.id}/thumbnail`;im.alt=e.title;const label=document.createElement('span');label.textContent=e.title;b.append(im,label);b.addEventListener('click',()=>{for(const old of document.querySelectorAll('.example'))old.classList.remove('active');b.classList.add('active');request(`/api/example/${e.id}`);});});$('examples').append(b);}const test=evidence.models[evidence.deployment].test,t=evidence.triage.test;for(const [value,label] of [[test.accuracy,'Raw known-shot accuracy'],[t.known_coverage,'Known shots handled automatically'],[t.known_automatic_accuracy,'Accuracy on automatic known shots'],[t.out_of_scope_review_rate,'Out-of-scope shots sent to review']]){const box=document.createElement('div');box.className='metric';const number=document.createElement('strong'),caption=document.createElement('span');number.textContent=value===null?'—':pct(value);caption.textContent=label;box.append(number,caption);$('metrics').append(box);}$('evidence-note').textContent=`${test.n} known-shot test clips and ${t.n_out_of_scope} out-of-scope clips. Automatic accuracy across the complete mixed test set: ${pct(t.automatic_accuracy_all)}. Threshold and calibration were fixed before this run's test evaluation. Existing benchmark sources were evaluated in the earlier study.`;}catch{$('evidence-note').textContent='Benchmark files are not available. Follow the project setup instructions.';}}
load();
```

templates/index.html

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>CricketShot / Six families</title><link rel="stylesheet" href="/static/app.css"><script src="/static/app.js" defer></script></head>
<body>
<header><a class="wordmark" href="/">Cricket<span>Shot</span></a><span class="local">Local video recognition / EE604</span></header>
<main>
  <section class="hero"><p class="eyebrow">SIX SHOT FAMILIES</p><h1>Recognise the<br><em>batting shot.</em></h1><p class="intro">One supported label when the evidence is clear. Human review when no class, or more than one class, passes the threshold.</p><p class="labels">Drive · Defence · Flick · Pull · Square cut · Sweep</p><p class="muted">Drive combines cover and straight drives. Other shot types are outside these six output families.</p></section>
  <div class="workspace">
    <section class="input-section"><h2><span class="step">01</span> Choose a batting interval</h2>
      <form id="upload-form"><label class="dropzone" for="video-input" id="dropzone"><span class="plus">+</span><strong id="file-label">Drop your video here</strong><span>or click to browse</span><small>MP4, AVI, MOV, MKV, WebM / below 100 MB</small></label><input type="file" id="video-input" accept="video/*,.avi,.mkv" hidden>
        <div id="trim-panel" hidden><video id="video-preview" controls preload="metadata" aria-label="Preview your selected video"></video><p id="preview-note" class="muted">Select the preparation, stroke and follow-through. Analyse one interval up to five seconds.</p><div class="time-controls"><label>Start (seconds)<input id="start" name="start" type="number" min="0" max="120" step="0.05" value="0"></label><label>End (seconds)<input id="end" name="end" type="number" min="0" max="120" step="0.05" placeholder="Clip end"></label></div><div class="time-buttons"><button type="button" id="mark-start">Use playback as start</button><button type="button" id="mark-end">Use playback as end</button></div></div>
        <button type="submit" class="primary-button" id="analyze-button" disabled>Analyze interval <span aria-hidden="true">→</span></button>
      </form><p class="muted input-note">A tightly trimmed shot around one second best matches the dataset. Use an end-on broadcast view with the batter visible. Longer uploads up to two minutes can be previewed and trimmed here.</p>
      <h3 class="examples-title">Dataset examples</h3><p class="muted">Selection notes and original labels accompany each result.</p><div class="examples" id="examples"></div>
    </section>
```

templates/index.html

```
16      <section class="result-section" aria-live="polite"><h2><span class="step">02</span> Read the
    decision</h2><div id="empty-state" class="empty-state"><strong>6</strong><h3>Follow the complete
    movement</h3><p>The decision, support scores and sampled frames will appear here.</p></div><div
    id="loading-state" class="loading-state" hidden><span class="spinner"></span><h3>Reading the batting
    interval</h3><p>Computing context, focused detail and review support…</p></div><div id="error-state"
    class="error-state" role="alert" hidden></div>
17        <div id="prediction" hidden><p class="eyebrow" id="decision-label"></p><h3 id="shot-label"
    class="shot-label"></h3><p id="decision-note"></p><p id="example-note" class="muted"></p><div
    class="threshold-note" id="threshold-note"></div><div id="scores" class="scores"></div><p class="muted">These
    support scores include the model's estimate that the interval belongs to the six supported families. They need
    not sum to 100%.</p>
18          <div id="human-review" class="human-review" hidden><h4>Record a human decision</h4><p>Inspect the
    movement before choosing a label. The model has left this case unresolved.</p><label>Reviewed label<select
    id="review-label"><option value="">Choose after inspection</option><option value="drive">Drive</option><option
    value="defence">Defence</option><option value="flick">Flick</option><option value="pull">Pull</option><option
    value="square_cut">Square cut</option><option value="sweep">Sweep</option><option
    value="outside_or_unclear">Outside these categories / unclear</option></select></label><label>Optional
    observation<textarea id="review-notes" rows="2" maxlength="500"></textarea></label><button id="download-review"
    class="secondary-button" disabled>Download reviewed record</button><p class="muted">The record stays local. A
    review does not silently retrain the model.</p></div>
19          <div class="frames-title"><h4>Sampled model inputs</h4><span id="timing" class="muted"></span></div><p
    class="muted">Focused view</p><div id="focused-frames" class="frame-strip"></div><details><summary>Full view and
     activity</summary><div id="full-frames" class="frame-strip"></div><canvas id="activity-chart" width="800"
    height="220" aria-label="Raw and camera-compensated activity"></canvas><p id="motion-note"
    class="muted"></p></details><p id="frame-note" class="muted"></p>
20          <details><summary>Score explanation</summary><p id="scope-note"></p><p>The six conditional class scores
    describe the model's choice among its supported labels. Multiplying by the scope score gives the support used
    for thresholding. A high score can still be wrong.</p><div id="conditional-scores"
    class="conditional-scores"></div><p class="muted">Neither the scope model nor the threshold guarantees detection
     of every unusual shot.</p></details><button id="download-result" class="text-button">Download prediction
    JSON</button>
21        </div>
22      </section>
23    </div>
24    <section class="evidence"><p class="eyebrow">HELD-OUT SOURCE GROUPS</p><h2>Accuracy and review
    workload</h2><p>Raw accuracy includes every supported-shot test clip. Automatic accuracy is measured only on
    cases accepted by the policy; deferred cases are not credited as correct.</p><div id="metrics"
    class="metrics"></div><p id="evidence-note" class="muted"></p><details><summary>Method and
    limits</summary><p>Frozen CNNs provide full-video context and focused frame detail. Train-only dimensionality
    reduction and regularised heads learn six shot families. A binary scope model learns from separate non-target
    stroke examples. Calibration and threshold selection use different source groups.</p><p>Exactly one support
    score strictly above the fixed threshold gives an automatic label. Zero or multiple passing classes gives human
    review. Threshold targets are empirical development targets, rather than guarantees for new videos.</p><p>The
    benchmark reuses sources from the earlier eight-class study. Source grouping reduces known leakage, but match
    identities and all replay footage have not been independently verified. Fixed crops, ambiguous labels and
    unfamiliar cameras remain limitations.</p><a href="{{ submission.github_url }}">Code, report and
    presentation</a></details></section>
25    <footer><span>EE604 / Cricket Shot Recognition<br>{% for member in submission.members %}{{ member.name }} ({{
    member.roll_number }}){% if not loop.last %} / {% endif %}{% endfor %}</span><span>Local inference after
    setup.<br>No cloud upload.</span></footer>
26  </main></body></html>
```

tests/test_intervals.py

```
1  """Interval provenance and full/focused preprocessing equivalence on lossless video."""
2  import cv2,numpy as np,pytest
3  from cricketshot.recognizer import decode_interval
4  from cricketshot.features import decode,decode_focused
5
6
7  def make_clip(tmp_path,n=120,fps=30):
8      path=tmp_path/'pattern.avi';writer=cv2.VideoWriter(str(path),cv2.VideoWriter_fourcc(*'FFV1'),fps,(160,96))
9      assert writer.isOpened()
```

tests/test_intervals.py

```
    for i in range(n):writer.write(np.full((96,160,3),[i*3%256,i*5%256,i*7%256],np.uint8))
    writer.release();return path


def test_interval_matches_original_training_preprocessing(tmp_path):
    path=make_clip(tmp_path);full,_=decode(path);focused=decode_focused(path)
    selected,detail,info,first=decode_interval(path,.5,1.5)
    assert first==15 and info['selected_frames']==30 and info['start_seconds']==.5 and info['end_seconds']==1.5
    np.testing.assert_array_equal(selected,full[15:45]);np.testing.assert_array_equal(detail,focused[15:45])


def test_longer_upload_requires_one_selected_interval(tmp_path):
    path=make_clip(tmp_path,n=61,fps=10)
    with pytest.raises(ValueError,match='Choose a start and end'):decode_interval(path)
    full,focused,info,first=decode_interval(path,1,2)
    assert len(full)==len(focused)==10 and first==10 and info['duration_seconds']==1


@pytest.mark.parametrize('start,end',[(-1,1),(1,0),(0,6),(float('nan'),1),(0,float('inf'))])
def test_invalid_interval_is_rejected_before_decoding(start,end):
    with pytest.raises(ValueError):decode_interval('not_a_file.avi',start,end)
```

tests/test_policy.py

```
"""Decision-boundary and numeric inference checks, independent of trained weights."""
import numpy as np
from cricketshot.policy import decide,calibrated_support,head_logits


def test_exactly_one_class_is_automatic():
    r=decide([.7,.1,.05,.05,.05,.05],.3)
    assert r['decision']=='automatic' and r['class']=='drive' and r['candidate_classes']==['drive']


def test_multiple_candidates_require_review():
    r=decide([.36,.35,.1,.06,.04,.04],.3)
    assert r['decision']=='human_review' and r['class'] is None and len(r['candidate_classes'])==2


def test_no_candidate_and_threshold_equality_require_review():
    r=decide([.3,.2,.2,.1,.1,.1],.3)
    assert r['decision']=='human_review' and r['candidate_classes']==[]


def test_scope_support_can_reject_a_confident_conditional_class():
    support,conditional,scope=calibrated_support(np.array([.99,.002,.002,.002,.002,.002]),-5,{'temperature':1,'scope_slope':1,'scope_intercept':0})
    assert conditional.argmax()==0 and conditional[0]>.98
    assert np.isclose(support.sum(),scope) and decide(support,.3)['class'] is None


def test_projection_and_relu_match_manual_arithmetic():
    h={'mean':np.array([1,2],np.float32),'scale':np.array([2,1],np.float32),'input_clip':np.array(5),'projection_center':np.array([0,0],np.float32),'projection':np.array([[1,2]],np.float32),'weight1':np.array([[2],[-1]],np.float32),'bias1':np.array([-1,1],np.float32),'weight':np.array([[1,3],[2,-1]],np.float32),'bias':np.array([.5,1],np.float32)}
    np.testing.assert_allclose(head_logits(np.array([3,3],np.float32),h),[5.5,11])
```

tests/test_web.py

tests/test_web.py

```
"""HTTP input handling and unresolved decisions remain separate from classifier accuracy."""
from pathlib import Path
import importlib.util,io,json
import numpy as np
import pytest
ROOT=Path(__file__).resolve().parents[1]
spec=importlib.util.spec_from_file_location('six_app',ROOT/'app.py');module=importlib.util.module_from_spec(spec);spec.loader.exec_module(module)
from cricketshot.policy import CLASSES,decide
from cricketshot.recognizer import decode_interval


class FakeRecognizer:
    def predict(self,path,start=0.,end=None):
        assert Path(path).name=='clip.avi'
        return decide([.2,.18,.17,.15,.12,.1],.3),{'full':np.zeros((24,40,80,3),np.uint8),'focused':np.zeros((24,40,40,3),np.uint8)}


@pytest.fixture
def client():return module.create_app(FakeRecognizer()).test_client()


def test_home_has_six_families_and_a_human_review_panel(client):
    response=client.get('/');assert response.status_code==200
    assert b'SIX SHOT FAMILIES' in response.data and b'Record a human decision' in response.data
    status=client.get('/api/status').get_json();assert status['classes']==6 and status['version']=='4.0.0'


def test_missing_or_unsupported_upload_is_rejected(client):
    assert client.post('/api/predict').status_code==400
    assert client.post('/api/predict',data={'video':(io.BytesIO(b'x'),'clip.exe')},content_type='multipart/form-data').status_code==400


def test_uploaded_filename_does_not_control_path_or_force_a_label(client):
    r=client.post('/api/predict',data={'video':(io.BytesIO(b'test bytes'),'../../outside.avi')},content_type='multipart/form-data')
    assert r.status_code==200
    data=r.get_json();assert data['class'] is None and data['decision']=='human_review' and set(data['support_scores'])==set(CLASSES)


def test_upload_size_limit_is_enforced():
    app=module.create_app(FakeRecognizer());app.config['MAX_CONTENT_LENGTH']=100
    r=app.test_client().post('/api/predict',data={'video':(io.BytesIO(b'x'*200),'clip.avi')},content_type='multipart/form-data')
    assert r.status_code==413 and '100 MB' in r.get_json()['error']


def test_invalid_interval_number_is_a_user_error(client):
    r=client.post('/api/predict',data={'video':(io.BytesIO(b'x'),'clip.avi'),'start':'bad'},content_type='multipart/form-data')
    assert r.status_code==400


def test_corrupt_video_is_a_user_error():
    class Decoder:
        def predict(self,path,start=0.,end=None):return decode_interval(path,start,end)
    r=module.create_app(Decoder()).test_client().post('/api/predict',data={'video':(io.BytesIO(b'not video'),'clip.avi')},content_type='multipart/form-data')
```

tests/test_web.py

```
  54      assert r.status_code==400
```